

EXECUTABLE SCIENCE

How AI Will Turn Research into Versioned, Verifiable Systems

Sînică Alboaie

Working manuscript · Version 0.5 July 2026

Axiologic Research SRL · ACHILLES research programme

Science should not become a factory for plausible papers. It should become an infrastructure in which claims can be executed, inspected, challenged, revised, and responsibly released.

A book about scientific automation after the manuscript

Working manuscript

This edition is a research and editorial manuscript, not a product specification or a claim that the proposed architecture has achieved universal scientific autonomy.

The manuscript distinguishes demonstrated systems, architectural inference, the ACHILLES research direction, and explicit scenarios for 2026-2040. Exact empirical claims are attributed in the text and references.

The abstract principles of the author's article in preparation, "Automated R&D and Mechanical Minds," are integrated without reproducing its illustrative examples.

Copyright © 2026 Sînică Alboaie. Working copy for scholarly review.

Contents

Abstract	4
Editorial and epistemic note	5
Prologue: When the Paper Is No Longer the System of Record	6
Chapter 1. Generative Abundance and the Validation Economy	9
1.1 The cheapening of candidate science	9
1.2 What remains scarce: criticism, causality, and accountable authority	11
1.3 Mechanical loops and meta-rational science	13
Chapter 2. From Publication to an Executable Research Object	16
2.1 Science becomes software-like, but not software	16
2.2 Version control for claims, evidence, and reasons	18
2.3 The paper as a signed view: transparency, privacy, and public argument	20
Chapter 3. Specification-Driven Inquiry	23
3.1 The living research specification, the Specification Ladder, and the Mirroring Pattern	23
3.2 Specification explosion and portfolio science	25
3.3 Hybrid execution: deterministic pipelines, bounded agents, and selective recomputation	27
Chapter 4. The Architecture of the Scientific Workbench	30
4.1 Local autonomy, global evidence	30
4.2 From industrial skills to scientific interpreters and MRP-VM	32
4.3 Independent critics, evidential escalation, and decision rights	34
Chapter 5. Methodological Rigor Across the Sciences	37
5.1 Formal and computational sciences: strong oracles, fragile formalizations	37
5.2 Physical and life sciences: instruments, calibration, and resistance from the world	38
5.3 Clinical, social, and interpretive sciences: causality, legitimacy, and plural evidence	40
Chapter 6. The Human and Institutional Transition	43
6.1 From vibe coding to vibe research: the accumulation of epistemic debt	43
6.2 Deskillling, credit, peer review, and scientific monoculture	44
6.3 Governance, privacy, and the legitimate role of slowing down	46
Chapter 7. What Is Possible, What Is Probable, and What Is Desirable	48
7.1 What is possible now: the scientific copilot as an evidence engineer	48
7.2 What is probable: 2030, 2035, and 2040	50
7.3 What is desirable: a concrete programme for executable science	52
Epilogue. Science After the Manuscript	56
References	58

Abstract

Artificial intelligence is lowering the cost of many operations that once limited the pace of research. Systems can search and synthesize literature, propose hypotheses, write and execute programs, design experiments, analyse results, operate laboratory equipment, generate formal proofs, and produce manuscripts. This progress is real, but its most important consequence is not that an artificial scientist has arrived. It is that candidate science is becoming abundant before justification, causal attribution, specialist attention, and institutional responsibility have become equally scalable.

This book argues that the durable response is to make science increasingly executable. Executability does not mean reducing inquiry to software or claiming that every form of knowledge can be formalized. It means representing enough of a research process in machine-operational form that its transformations, dependencies, evidence obligations, and decision rights can be inspected and, where appropriate, rerun. A paper then ceases to be the sole system of record. It becomes a signed human argument over a larger, versioned research object containing specifications, claims, data, code, protocols, models, negative results, reviews, provenance, uncertainty, and the history of consequential revisions.

The central architectural proposal is specification-driven inquiry. A persistent research specification records objectives, admissible claim types, assumptions, invariants, methods, evaluators, evidence thresholds, resource budgets, stopping conditions, risks, and authorities. Instead of compiling into one linear plan, it undergoes specification explosion: controlled expansion into competing hypotheses, representations, baselines, negative controls, causal ablations, scaling tests, and refutation attempts. Deterministic pipelines execute stable transformations; bounded agents explore open branches; symbolic and neuro-symbolic interpreters provide stronger checks where structure permits them; independent critics attach typed evidence; and accountable humans decide whether claims should be promoted, narrowed, held, or released.

This architecture generalizes several currents already visible in industry and academia. Structured generation, language-model programming, agent frameworks, model context protocols, coding skills, specification-driven development environments, autonomous laboratories, theorem proving, program search, and machine-assisted peer review all supply partial mechanisms. The ACHILLES research direction considered here proposes a unifying layer: a Specification Ladder and Mirroring Pattern for long-lived artefact chains; SOP Lang for plans, and pipelines; generalized skills as bounded execution regimes; and an emerging MRP-VM runtime for regime-sensitive routing, validation, and reintegration. These are presented as research hypotheses and engineering directions, not as a mature universal platform.

The book also examines the methodological and human limits of the analogy with software. Formal and computational fields can often use strong executable oracles, but correctness depends on faithful formalization. Physical and life sciences require instruments, calibration, controls, and resistance from the world. Clinical and social sciences add causal, ethical, institutional, and distributional obligations. Interpretive disciplines require transparent argument and contestable evidence rather than a false promise of algorithmic closure. Across all domains, the lesson is the same: the appropriate degree of autonomy depends on the strength, latency, and independence of validation.

The probable future is neither a universal autonomous scientist nor a return to unaided research. It is a layered scientific workbench in which local autonomy is compatible with global evidence and named authority. The desirable future is one in which automation increases the rate at which weak claims are rejected, important alternatives are explored, results are reproduced, and

scarce human judgment is allocated well. The dangerous future is one in which paper production, benchmark optimization, and institutional metrics accelerate while understanding and responsibility decay. The choice between these futures is architectural and institutional, not merely a question of model capability.

Editorial and epistemic note

This manuscript combines four kinds of statements that must not be confused. Some are descriptions of demonstrated systems and published results. Some are architectural inferences from those results. Some are proposals associated with the ACHILLES research programme. Others are explicitly speculative forecasts about the period from 2026 to 2040. The text marks the difference through its language. “Possible now” refers to capabilities that can be assembled from currently demonstrated components, although not necessarily with production reliability. “Probable” refers to developments that follow from observable technical and institutional trajectories but remain contingent. “Desirable” refers to a normative position: what should be built if scientific automation is to strengthen rather than merely accelerate inquiry.

The book develops and generalizes principles first formulated in the author’s open letter in preparation, “Automated R&D and Mechanical Minds.” Only its abstract methodological claims are carried forward here. The domain-specific exploratory anecdotes used in that article are intentionally omitted. The purpose of this manuscript is not to expand one case study, but to formulate an architecture and institutional theory of executable science.

The book uses “science” broadly but not carelessly. It includes formal proof, computational experimentation, laboratory research, observational inference, clinical investigation, social inquiry, and interpretive scholarship. These activities do not share one oracle of truth. Executable science therefore means domain-sensitive operationalization: enough explicit structure to make obligations and transformations inspectable without pretending that all judgments can be delegated to computation.

Prologue: When the Paper Is No Longer the System of Record

For several centuries, the scientific article has been asked to perform too many functions at once. It reports a question, compresses a method, presents selected evidence, advances an interpretation, claims novelty, assigns authorship, invites criticism, and becomes the unit by which institutions allocate reputation and resources. The article is an extraordinary social technology. It turns dispersed labour into a public argument that can be read, contested, cited, and preserved. Yet its success has also hidden a structural weakness: the manuscript is a lossy projection of the research process, not the process itself.

The loss was tolerable while most transformations were slow and human attention was the dominant production constraint. A small research team could reconstruct why an analysis changed, which failed experiment redirected the project, or which informal judgment led to a new model. Code, laboratory notebooks, email, instrument files, preregistrations, and draft manuscripts were fragmented, but the people involved often retained a working narrative. When a paper was released, the compression was accepted as a practical necessity. Reviewers could request additional detail, later researchers could attempt replication, and controversies could unfold over years.

Generative and agentic AI change this balance. A system can now create more candidate hypotheses, code paths, simulations, analyses, visualizations, and prose than a human team can understand in the same interval. In computational domains, an agent can modify a repository, run experiments, interpret outputs, and rewrite a manuscript in a continuous loop. In experimental domains, software can connect literature, planning, active learning, robotics, and measurement. In formal domains, a language model can propose constructions while a theorem prover or programmatic evaluator rejects invalid candidates. The bottleneck moves. Production becomes cheaper; reconstruction and justification become more expensive.

This shift exposes the inadequacy of treating the final paper as the only durable object. Suppose an automated system generates twenty plausible mechanisms, implements eight, finds three positive benchmark effects, eliminates one through a negative control, narrows another after a scaling failure, and retains a third only under a revised claim. A conventional paper may present the surviving line as a clean story. The polished narrative can be honest, but it conceals the branching structure that gives the result meaning. It does not show whether the discarded mechanisms were genuinely tested, whether the evaluator changed, whether the same model family generated and judged the evidence, or whether a specialist objection altered the interpretation. At machine speed, those omissions become operational risks rather than merely historiographical losses.

The analogy with software engineering is therefore more than rhetorical. Modern software is not maintained as a sequence of published screenshots. It lives in repositories. Requirements, source code, tests, dependency declarations, issues, review comments, continuous-integration results, releases, and change histories remain connected. A developer can inspect what changed, who approved it, which tests passed, and which version entered production. Branches support exploration without erasing the main line. A broken dependency can trigger selective rebuilding. The artefact is not only the executable program; it is the governed history that makes the program maintainable.

Scientific knowledge will increasingly need a comparable infrastructure, although its semantics are more demanding. A scientific repository must version more than files. It must version claims, evidential status, assumptions, models, operational definitions, causal interpretations, review

obligations, and decision rights. A commit should not merely record that a notebook changed. It should record that a claim was narrowed because a negative control succeeded, that an evaluator was replaced because it could not distinguish competing mechanisms, or that an apparent improvement disappeared under matched compute. The unit of change is epistemic as well as textual.

This is the central meaning of executable science. The phrase does not imply that a computer can decide truth by running a script. It means that a larger part of the path from question to claim is represented as an inspectable system. Some components are executable in the strict sense: code, workflows, proofs, simulations, statistical models, instrument procedures, and validation rules. Others are machine-readable commitments: the objective of the inquiry, the type of claim being made, the evidence required, the scope of a reviewer's judgment, the authority that may release a result, and the conditions under which a branch must stop. Still others remain human acts of interpretation, value judgment, and responsibility. Executability is therefore layered. It is the discipline of making each layer explicit enough that its powers and limits are visible.

The likely successor to the paper-centric model is an evidence-bearing research object. This object has a stable identity and versioned history. It contains a graph of claims and a graph of evidence, linked to data, code, protocols, environments, instrument records, models, reviews, and known failures. It can generate several views. The article is the public argument. A reviewer receives a packet focused on the claims requiring judgment. A replicator receives executable environments and divergence reports. A regulator receives a dossier of authority, risk, and provenance. A future research agent receives a compact record of what was tried, why it failed, and which uncertainties remain open. None of these views is the whole object, and no view should silently overwrite the others.

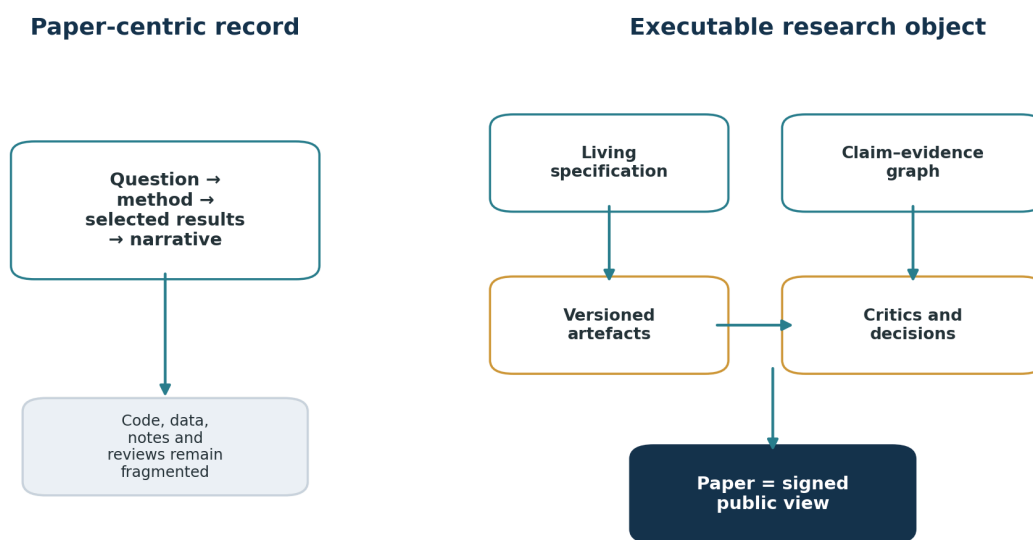
This transformation would alter the economics of science. Today, the visible product is usually the positive paper, while the expensive negative result, the careful replication, the maintained benchmark, the repaired dataset, and the decisive causal ablation often receive less credit. A versioned research object can make these contributions attributable. It can show that a branch was closed before a costly experiment, that a reviewer identified a common-mode failure, or that an engineer made a result reproducible across environments. The infrastructure would not automatically create fair incentives, but it would make invisible epistemic labour more legible.

The transformation would also create dangers. A complete research log can become a surveillance system. Early speculation, private disagreement, failed attempts, and provisional ideas are essential to inquiry. If every unformed thought becomes permanent evidence in an employment or funding decision, researchers will optimize for defensibility rather than discovery. Executable science therefore requires layered transparency. The public deserves evidence for consequential claims, but the researcher requires a protected space in which to be wrong. What must be preserved is not an indiscriminate transcript of every interaction. It is the history of reasons that changed the public state of the inquiry.

The book's argument unfolds from this premise. Chapter 1 examines generative abundance and the new validation economy. Chapter 2 defines the research object and the analogy with version-controlled software. Chapter 3 develops specification-driven inquiry, including the Specification Ladder, Mirroring Pattern, specification explosion, and the hybrid use of pipelines and agents. Chapter 4 proposes the architecture of the scientific workbench and distinguishes existing industrial mechanisms from the more speculative ACHILLES runtime. Chapter 5 asks what computational verifiability can mean across different sciences. Chapter 6 examines the human and institutional transition by extrapolating from coding agents, including hidden debt, deskilling, peer review, credit, and legitimate slowing. Chapter 7 separates what is possible, probable, and desirable, then formulates a concrete research programme and falsifiable predictions.

The message is not that science will become software. Science is answerable to mathematical structure, empirical resistance, causal identification, human meaning, and public consequence in ways that software is not. The message is that science will become more software-like in its operational form. Its results will increasingly live in versioned systems rather than isolated documents. Its claims will be connected to executable tests and provenance. Its revisions will be localized and reconstructable. Its agents will work inside governed runtimes rather than unbounded conversations. Its institutions will have to decide which parts of judgment can be automated, which must remain human, and how disagreement can remain visible when machines produce persuasive consensus at scale.

The decisive question is therefore not whether a model can imitate a scientist. It is whether scientific institutions can become computationally articulate without becoming mechanically minded. A laboratory that can generate indefinitely but cannot explain why a claim changed has automated activity, not inquiry. A repository that preserves every token but not the reasons for a decision has archived noise, not knowledge. An agent that passes internal tests but cannot expose its assumptions has produced performance, not authority. Executable science begins when the system is designed to remember these distinctions.



The durable object is the governed history, not the final document alone.

Figure 1. From a paper-centric record to an evidence-bearing research object. The paper remains a signed public argument, while the durable system of record preserves specification, artefacts, evidence, criticism, and change.

Chapter 1. Generative Abundance and the Validation Economy

1.1 The cheapening of candidate science

The strongest near-term effect of AI on research is not the replacement of scientists. It is the radical reduction in the cost of producing candidate scientific work. A candidate may be a literature synthesis, a mechanism, an equation, a proof sketch, a software implementation, an experiment plan, a statistical model, a design for an instrument, an interpretation of results, or a complete manuscript. Each of these objects once required a substantial interval of expert labour. Increasingly, they can be drafted, translated between representations, and iterated by systems that combine foundation models with retrieval, code execution, search, domain tools, and persistent memory.

This development continues a much older programme. The Logic Theory Machine demonstrated that proof search could be mechanized. DENDRAL showed that domain knowledge could constrain scientific hypothesis formation in chemistry. Robot Scientist systems linked hypothesis generation to laboratory experimentation and falsification in bounded biological domains (Newell and Simon, 1956; Lindsay et al., 1993; King et al., 2004, 2009). Program-search systems have since discovered faster matrix multiplication algorithms and new mathematical constructions, while autonomous chemistry and materials platforms have joined planning, execution, and measurement in closed or semi-closed loops (Fawzi et al., 2022; Romera-Paredes et al., 2024; Boiko et al., 2023; Szymanski et al., 2023). What language models add is a general translation layer. They can move between ordinary language, scientific prose, code, symbolic notation, database queries, and tool calls without requiring a handcrafted interface for every transition.

Recent systems make the changing frontier visible. The AI Scientist integrates ideation, literature search, code modification, experiment execution, analysis, manuscript production, and automated review in a machine-learning setting (Lu et al., 2026). Google's Co-Scientist uses a multi-agent search-and-evolution process to generate and refine biomedical hypotheses under scientist-specified goals, with selected proposals subjected to laboratory validation (Gottweis et al., 2026). FutureHouse's Robin combines literature agents and data-analysis agents in an iterative biology workflow that connects hypotheses to experimental results (Ghareeb et al., 2026). These systems differ in architecture and evidential strength, but they demonstrate that substantial portions of the research cycle can be represented as computational operations.

The same pattern appears in formal and computational discovery. FunSearch couples a language model to an evaluator and an evolutionary archive, allowing the model to propose program fragments while an executable function determines whether they are valid and how well they perform (Romera-Paredes et al., 2024). AlphaEvolve generalizes the idea toward algorithm discovery and optimization by combining generative models, program mutation, evaluators, and an evolving pool of candidates (Novikov et al., 2025). AlphaGeometry and related neuro-symbolic theorem-proving systems use language models or learned components to propose constructions while symbolic engines enforce proof obligations (Trinh et al., 2024). Their success is not evidence that free-form language models have acquired infallible mathematical judgment. It is evidence that generation becomes powerful when embedded in a search process with a strong external evaluator.

Laboratory automation supplies another route. ChemOS, self-driving laboratories, A-Lab, automated optical platforms, and emerging multi-agent chemistry systems demonstrate that planning can be coupled to instruments through machine-readable protocols, active learning, Bayesian optimization, and recovery procedures (Häse et al., 2019; Burger et al., 2020; Szymanski

et al., 2023; Panapitiya et al., 2026). Physical automation introduces a kind of resistance that purely textual loops lack. A synthesis fails, a sensor drifts, an instrument saturates, or a material behaves differently from the model. Yet this resistance is useful only when the system records calibration, uncertainty, execution context, and deviations. A robot that repeats an invalid assay at scale has not made the research more rigorous.

Information-centred systems are likely to spread fastest because their inputs and outputs are already digital. Tools such as Elicit, Semantic Scholar, Scite, PaperQA2, and STORM automate parts of search, evidence extraction, citation-aware synthesis, and document construction. Their practical value is considerable, particularly when they reduce the cost of mapping a literature, comparing claims, or identifying missing evidence. But an information workflow is not automatically a scientific workflow. Search quality, corpus access, source reliability, claim granularity, citation fidelity, and the treatment of negative results all determine whether the output supports inquiry or merely produces a fluent review.

The relevant distinction is not between tasks that are intrinsically human and tasks that are intrinsically mechanical. It is between operations that can be delegated under an explicit contract and decisions whose validity or legitimacy depends on context, external resistance, specialist judgment, or consequence. Data conversion can often be deterministic. A literature query can be delegated if scope and source policy are declared. Code can be generated if tests and environment constraints are explicit. A hypothesis can be proposed by a model. Whether a question is worth asking, whether an operationalization is faithful, whether a negative result closes a branch, whether a causal interpretation is warranted, and whether a finding should influence patients or public policy require more than syntactic competence.

As the cost of generation falls, the traditional relationship between proposal and attention reverses. In the old regime, a research group might explore a small number of hypotheses because each implementation and experiment was expensive. The scarcity of production indirectly filtered the space. That filter was crude and often excluded unconventional ideas, but it prevented unlimited branching. In the emerging regime, a system can formulate hundreds of plausible directions before a specialist has read the first ten. The constraint shifts from the supply of ideas to the capacity to discriminate among them.

This shift is already visible in benchmarks. PaperBench decomposes the replication of twenty machine-learning papers into thousands of gradable tasks and shows that even capable agents remain far from reliable end-to-end reproduction (Starace et al., 2025). MLE-bench measures the ability to complete machine-learning engineering tasks under competition-like conditions and similarly reveals a gap between local coding competence and sustained project execution (Chan et al., 2024). DiscoveryWorld and laboratory-oriented benchmarks expose failures in experimental reasoning, tool use, and state tracking (Jansen et al., 2024; Laurent et al., 2024). These evaluations do not define science, but they demonstrate a recurring pattern: generating a plausible next action is much easier than maintaining a valid chain from objective to result.

The consequence is a new production function for research. More model capability and more test-time compute can increase the volume and sometimes the quality of candidate outputs. The AI Scientist reports improvements with deeper experimental search; Co-Scientist reports improvements as hypotheses undergo extended generation, debate, ranking, and evolution (Lu et al., 2026; Gottweis et al., 2026). Yet scaling search does not itself guarantee that the evaluator measures the right property. If the reward is workshop acceptability, the system may learn the form of a workshop paper. If the reward is benchmark score, it may find a benchmark-specific shortcut. If novelty is estimated through semantic distance, it may produce unfamiliar phrasing rather than a new mechanism. Generative abundance therefore increases the importance of evaluator design rather than reducing it.

A useful way to describe the transition is that science is moving from an economy of scarce conjecture to an economy of scarce validation. The phrase should not be taken literally in every field. Many good hypotheses have always been plentiful, while access to data, instruments, participants, and expert time has long been scarce. What changes is the scale and speed at which plausible research objects can be synthesized. The ability to produce them no longer indicates that the surrounding evidence can support them.

The first design principle of executable science follows. Research automation should be evaluated by the quality-adjusted rate at which claims acquire independent support or are efficiently rejected, not by the rate at which hypotheses, experiments, or papers are generated. This principle changes what should be optimized. A system that produces fewer claims but identifies decisive counterexamples earlier may be more valuable than one that fills a repository with polished results. A workbench that makes a hidden assumption visible may create more scientific value than a model that adds another percentage point to an internal score.

Table 1. The emerging validation economy

Functions becoming abundant	Functions that remain scarce
Literature retrieval, synthesis, and reformulation	Judging which omissions, assumptions, and contradictions are consequential
Generation of hypotheses, mechanisms, code paths, models, and prose	Causal attribution, discriminating experiments, and the design of decisive negative evidence
Execution of simulations and digitally instrumented workflows	Independent reproduction across tools, laboratories, datasets, and theoretical frames
Production of polished figures, reports, and manuscripts	Specialist attention, accountable release, and institutional willingness to own a claim
Repeated self-critique by the same model family	Criticism with genuinely different information, incentives, methods, or authority

1.2 What remains scarce: criticism, causality, and accountable authority

Generative systems make certain kinds of intellectual production cheap, but they do not make every epistemic function cheap. The most important scarcities migrate toward the work that determines what a result means and whether anyone should rely on it. Three forms of scarcity are especially consequential: decisive criticism, causal attribution, and accountable authority.

Decisive criticism is not generic negativity. It is the design of a test that can discriminate between live alternatives. A weak critic restates limitations or assigns an impressionistic score. A strong critic identifies the observation that would force a claim to be narrowed, the component whose removal would reveal a simpler explanation, the scaling regime in which a local property ceases to matter, or the external dataset on which an apparent effect should survive. Such criticism requires knowledge of the domain and of the specific claim type. A theorem claim needs proof checking and faithful formalization. A mechanism claim needs interventions or discriminating observations. A comparative performance claim needs matched baselines, budgets, and variance analysis. A safety claim needs hazard identification, adversarial testing, and a consequence-sensitive threshold.

Automated critics can help. Models can search for prior work, compare manuscripts with code, generate test cases, execute robustness checks, detect citation mismatches, or propose counterexamples. The randomized deployment of an AI feedback agent at ICLR 2025 showed that machine-generated suggestions could make a substantial fraction of peer reviews more specific and actionable, without transferring the final editorial decision to the system (Thakkar et al., 2026). This is a promising pattern because it uses AI to strengthen human criticism rather than to

manufacture an automated verdict. The critic's output remains inspectable, and the reviewer remains responsible for accepting or rejecting it.

The independence of criticism matters as much as its sophistication. If one model family proposes a claim, writes the implementation, selects the evidence, interprets the result, and evaluates the manuscript, apparent agreement may be a common-mode error. Different agents with different prompts are not necessarily independent if they share training data, retrieval sources, code, evaluators, and institutional incentives. Executable science therefore needs dependency fingerprints. A review object should record which models, datasets, prompts, tools, and human operators contributed to each judgment. Independence should be treated as an evidential property, not inferred from the number of personas in a multi-agent conversation.

Causal attribution is a second scarcity. Research systems can find correlations, optimize outcomes, and construct elaborate explanations, but scientific value often depends on assigning the observed effect to the right cause. When a complex method improves a benchmark, the improvement may come from more compute, a data leak, a changed preprocessing step, a stronger baseline implementation, or an evaluator that sees only a proxy. The appropriate response is not merely to rerun the experiment. It is to design ablations and controls that transfer causal credit from the overall system to the components that matter.

This is where specification-driven automation can outperform unaided workflow. Once the claim is typed, the workbench can associate it with standard obligations. A representation claim may require a simpler-carrier test. An architecture claim may require matched capacity and data. A causal policy claim may require an explicit estimand, identification assumptions, sensitivity analysis, and alternative causal graphs. A clinical claim may require pre-specified endpoints, adverse-event monitoring, subgroup analysis, and external validation. The workbench cannot determine all these obligations universally, but it can make them explicit and prevent a successful local score from silently expanding into a stronger claim.

Accountable authority is the third scarcity. Scientific claims are not all equally consequential. A conjecture released as a speculative note imposes different obligations from a recommendation that changes clinical care, environmental policy, or critical infrastructure. Even if an AI system could estimate evidential strength, it could not legitimately grant itself social permission to act. Release and deployment require a named person or institution that has the legal, professional, and ethical authority to accept responsibility.

Authority should not be confused with truth. An authorized institution can be wrong, captured, or negligent. The point is procedural: consequential decisions must remain contestable and attributable. An evidence-bearing system should record who defined the objective, who approved the method, who reviewed the result, who authorized release, and under which policy version. It should also record the procedure by which a claim can be challenged, corrected, superseded, or withdrawn. In a high-speed research environment, the ability to stop or reverse propagation becomes part of scientific reliability.

This perspective changes the role of human experts. The human is not valuable merely because humans are slower or because tradition requires a signature. Expertise becomes most valuable at points where evaluators are weak, stakes are high, or representations are contested. A statistician may inspect identification and uncertainty. A domain scientist may judge whether an assay captures the intended phenomenon. A systems expert may test operational scaling. A patient or community representative may identify harms omitted by a technical objective. A research-integrity officer may decide whether provenance and disclosure are sufficient. No participant needs omniscience, but the scope of each judgment must be explicit.

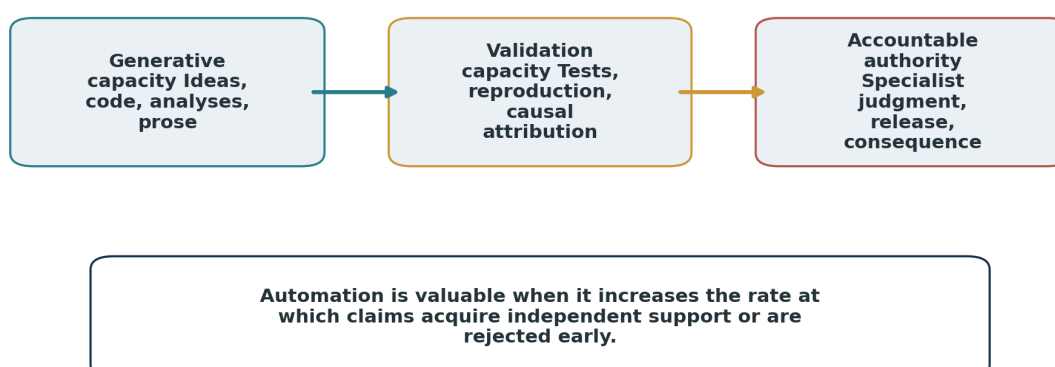
The same scarcity appears at the level of the scientific system. Hao and colleagues' analysis of more than forty million papers reports an association between AI use and increased individual output and impact, alongside a contraction in the collective range of topics studied (Hao et al., 2026). The result should be interpreted cautiously because observational classification cannot establish every causal mechanism. Nevertheless, it illustrates a plausible institutional effect: AI may accelerate work where data, benchmarks, and existing methods make evaluation easy, while drawing attention away from questions that are poorly instrumented or conceptually immature. Individual productivity can rise while collective exploration narrows.

The validation economy therefore includes attention allocation. Specialist time, laboratory access, participant risk, high-quality datasets, and public trust are limited resources. A responsible automated system should optimize the expected information value of consuming them. Before requesting an expensive experiment, it should ask whether a cheaper simulation, negative control, formal check, or independent reanalysis could eliminate the branch. Before escalating a claim to a senior specialist, it should pass inexpensive validity gates and state the narrow unresolved question. Before publishing, it should estimate not only novelty but the review burden imposed on the community.

This suggests a more demanding productivity metric. Instead of papers per researcher or hypotheses per compute unit, one might measure human validation hours per surviving claim, time to the first valid counterexample, independent reproduction rate, the fraction of claims demoted by simpler explanations, compute and energy per externally supported result, or the expected information gain of the next experiment. No single metric should become a universal target; Goodhart's law applies with particular force to science. The point is to direct measurement toward the scarce functions rather than the abundant ones.

The practical implication is that automated science should be asymmetric. Generation can be broad and inexpensive. Promotion should be narrow and increasingly expensive. Early branches can be speculative, but evidential thresholds should rise as a claim approaches public authority or operational consequence. The architecture should make it easy to generate alternatives and hard to erase disagreement. It should make it cheap to abandon a weak branch and costly to promote a claim without new kinds of evidence.

The validation economy



Generation can scale broadly. Promotion should become progressively narrower and more evidence-intensive.

Figure 2. The validation economy. Generative capacity can scale faster than causal attribution, independent criticism, specialist attention, and accountable release.

1.3 Mechanical loops and meta-rational science

The phrase “mechanical mind” is often used as a synonym for machine intelligence. That usage hides the more important danger. A mechanical research loop is any process, human or artificial, that continues to optimize a local proxy after the proxy has diverged from the purpose of inquiry. A language model can do this by optimizing plausibility. A laboratory can do it by repeating an assay whose operational meaning has been lost. A benchmark community can do it by improving scores on a task that no longer discriminates useful capability. A researcher can do it by maximizing publications, and an institution can do it by rewarding whatever is easiest to count.

The problem therefore predates AI. Scientific institutions already contain rituals, proxy measures, publication biases, and incentives for positive, legible results. Smaldino and McElreath model how selection pressures can favour poor methods even without individual fraud (Smaldino and McElreath, 2016). Ioannidis describes conditions under which published findings are likely to be false, while the reproducibility movement documents the effects of flexible analysis, underpowered studies, incomplete reporting, and selective publication (Ioannidis, 2005; Nosek et al., 2015; Munafò et al., 2017). Generative AI can intensify these tendencies because it reduces the cost of producing the forms that institutions reward.

A mechanical loop is dangerous precisely because it can be locally competent. Every step may look reasonable. The literature agent finds relevant papers. The planner proposes an experiment. The coding agent implements it. The evaluator reports improvement. The reviewer agent identifies no major issue. The manuscript becomes clearer with each revision. Yet the system may have silently changed the question, selected an evaluator that rewards the preferred mechanism, ignored a simpler explanation, or exhausted its budget on a branch that no longer serves the original objective. Local coherence does not guarantee global fidelity.

The opposite capacity is meta-rational design. Ordinary reasoning asks what follows inside a representation, model, or method. Meta-rational design also asks why that representation is being used, what it excludes, which alternative could explain the observation, when the evaluator has become uninformative, and whether the level of abstraction remains appropriate. It treats the choice of model, tool, critic, and stopping rule as part of the research problem rather than as fixed infrastructure.

This idea is related to rational metareasoning, which asks how an agent should allocate limited computational resources among reasoning strategies, and to model pluralism, which rejects the assumption that one representation captures every scientifically useful aspect of a system (Lieder and Griffiths, 2017; Veit, 2020). No-Free-Lunch results provide a formal caution against expecting one optimizer to dominate across all possible problem structures (Wolpert and Macready, 1997). In practice, the lesson is not that every project requires maximum heterogeneity. It is that a research system should be able to detect when its current regime is failing and route the problem elsewhere.

A meta-rational scientific workbench therefore uses heterogeneous intelligence. Language models are strong proposers, translators, summarizers, and interface layers. Theorem provers and SMT solvers provide stronger guarantees under explicit formal assumptions. Computer algebra systems manipulate exact symbolic structures. Probabilistic programs represent uncertainty and latent structure. Causal models express interventions and identification assumptions. Simulators encode domain dynamics. Databases and knowledge graphs maintain structured evidence. Laboratory instruments provide physical observations. Humans contribute tacit knowledge, value judgments, contextual understanding, and accountable authority. The system is intelligent not because one component is universal, but because it can combine and revise these regimes.

Routing among regimes requires a task contract. The system must know whether exactness, reversibility, order sensitivity, uncertainty calibration, latency, memory, energy, interpretability,

privacy, or robustness is most important. A method that is superior under one contract may be inferior under another. A theorem-proved algorithm may be irrelevant if the formalization omits the operational constraint. A compact approximate model may be preferable to an exact but intractable representation. A large language model may be the best tool for discovering terminology across fields, while a deterministic parser is the better tool for checking a schema. The contract makes the trade-off inspectable.

Translations between representations are themselves scientific claims. Moving from a natural-language objective to a formal estimand, from a theory to a simulation, from a dataset to a statistical model, or from a mechanistic hypothesis to a laboratory assay can lose distinctions and introduce assumptions. An executable system should declare what is preserved, what is approximated, whether an inverse mapping exists, how uncertainty propagates, and which validation checks apply. This requirement is central to the Specification Ladder developed later in the book. The most consequential errors often occur not inside one representation but at the boundaries between them.

Meta-rational design also protects against decorative complexity. Generative AI makes it inexpensive to connect advanced mathematical language, elaborate multi-agent roles, and sophisticated models to almost any research problem. Some of these connections will be genuinely productive. Others will create explanations whose complexity exceeds their causal contribution. The correct response is not a universal preference for simplicity. It is a demand that added structure earn its place through discriminating evidence. A more complex representation should create stronger proof obligations, better predictions, lower cost, improved robustness, or an explanatory distinction that a simpler alternative cannot provide.

The institutional analogue is equally important. A funding programme, journal, or laboratory can become mechanically rational by optimizing its own metrics. Meta-rational institutions preserve the ability to revise evaluators, create protected spaces for unconventional work, and recognize when high output masks declining diversity or reliability. They treat governance as an evolving constitution rather than a static checklist.

The central thesis of this chapter can now be stated precisely. AI makes the generative and procedural components of science increasingly abundant. The scarce components are the design of decisive criticism, causal attribution, the allocation of specialist attention, and accountable authority. An architecture for automated research should therefore amplify meta-rationality rather than merely increase iteration speed. It should keep objectives, representations, evaluators, and decision rights revisable and visible. Otherwise, the most capable systems may become the most efficient mechanical minds science has ever built.

Table 2. What current systems demonstrate - and what they do not yet establish

Demonstrated boundary signal	Interpretation
PaperBench evaluates replication of 20 AI research papers through 8,316 scored tasks; the best tested agent averaged 21.0 percent.	End-to-end research execution is measurable, but faithful replication remains far from routine even in a digitally native field.
The AI Scientist demonstrated an integrated loop from idea generation to experiments and manuscript production, with one of three generated submissions accepted at an ICLR workshop.	A complete machine-mediated research trajectory is possible under bounded conditions; acceptance of an isolated output is not evidence of general scientific reliability.
The A-Lab autonomous materials platform performed 353 experiments and realized 36 of 57 target compounds over 17 days.	Closed-loop discovery can achieve meaningful laboratory throughput when synthesis, measurement, and evaluators are tightly instrumented.

Demonstrated boundary signal	Interpretation
A randomized study involving more than 20,000 reviews found that reviewers updated 27 percent of reports after receiving AI-generated feedback.	AI can improve human review as a critic and preflight mechanism without replacing the reviewer's epistemic and institutional authority.

Sources: *Starace et al. (2025)*; *Lu et al. (2026)*; *Szymanski et al. (2023)*; *Thakkar et al. (2026)*.

Chapter 2. From Publication to an Executable Research Object

2.1 Science becomes software-like, but not software

The claim that science will become more like software can be misunderstood in two opposite ways. The first misunderstanding is reductionist: if a research process can be represented computationally, then its conclusions are no more than program outputs. The second is romantic: because scientific judgment exceeds computation, software concepts have little relevance to inquiry. Both positions miss the architectural change. Science is not reducible to software, but scientific work is increasingly mediated by systems whose reliability depends on software-like disciplines of modularity, testing, versioning, dependency management, and controlled release.

Software engineering learned these disciplines because complex systems cannot be maintained as final products alone. A compiled binary does not reveal the requirements, source changes, tests, reviews, dependencies, and incidents that justify its use. The operational object is the repository and its governed history. Version control permits parallel exploration, merge review, reversion, and attribution. Continuous integration reruns tests when relevant components change. Package manifests identify dependencies. Issues and design records preserve unresolved obligations. Releases freeze selected states for external use. None of these mechanisms guarantees that the software serves a good purpose or contains no defect. They make change tractable and failure diagnosable.

Research faces an analogous problem, with a richer object of control. A paper is comparable to a release note and a signed design argument, not to the repository itself. The scientific repository must include datasets, code, environments, protocols, calibration records, preregistrations, models, analysis plans, notebooks, figures, review interventions, and the claims these artefacts are intended to support. More importantly, it must preserve relations that ordinary file versioning does not understand. Which result bears on which claim? Which assumption is required by which inference? Which negative control caused a mechanism claim to be narrowed? Which reviewer certified a formal property but not its empirical relevance? Which policy authorized access to sensitive data? Which uncertainty remains unresolved at release?

The appropriate unit is an evidence-bearing research object. It is “evidence-bearing” because every promoted claim carries links to the artefacts, checks, criticisms, and authorities that justify its current status. It is a “research object” because it contains a structured inquiry rather than a finished narrative. The object has a stable identity, versioned states, and an appendable history. It can branch when alternative hypotheses or analyses must coexist. It can merge when independently developed evidence becomes compatible. It can mark a claim as superseded without deleting the reason it was once accepted.

Existing infrastructure provides important foundations. Reproducible research links analysis to code and data rather than presenting results as prose alone (Peng, 2011). Jupyter notebooks combine narrative and executable computation, although hidden state and environment drift can limit reliability (Kluyver et al., 2016). Git can improve transparency and reproducibility by preserving changes and collaboration history, but it versions files rather than epistemic relations (Ram, 2013). W3C PROV represents entities, activities, and agents; RO-Crate packages research data, software, workflows, and contextual metadata; FAIR principles encourage findability, accessibility, interoperability, and reuse (Lebo et al., 2013; Wilkinson et al., 2016; Soiland-Reyes et al., 2022). Workflow standards such as the Common Workflow Language can describe executable computational pipelines. None of these should be discarded. The missing layer is a common profile

that connects them to claim types, evidential stages, critic verdicts, unresolved obligations, and decision rights.

The word “executable” should therefore be applied with precision. A computational experiment may be executable in the literal sense: an environment can be reconstructed and a workflow rerun. A wet-laboratory protocol may be machine-executable only where instruments and materials are digitally controlled; elsewhere, it may be operationally specified but require human execution. A statistical claim can be re-estimated from data, yet its identification assumptions cannot be established by execution alone. An interpretive claim may have no deterministic evaluator, but its evidence, quotations, coding decisions, and argumentative dependencies can still be represented and contested. Executability is a gradient of operational explicitness, not a binary property.

The analogy with software is strongest at the level of maintenance. Science changes. New data arrive, a library is updated, an instrument calibration is corrected, a confound is discovered, or an external team fails to reproduce a result. Under the paper-centric model, the response may be a correction, a new paper, or an informal change in community belief. The relation between the old and new states is often difficult to reconstruct. Under a versioned research-object model, a change invalidates the downstream claims and artefacts that depend on it. The workbench can identify the affected subgraph, recompute what remains executable, request human review where interpretation changes, and release a new view with a clear divergence report.

This form of selective recomputation is one of the most practical advantages of the architecture. Suppose a preprocessing error affects one dataset but not the theoretical analysis or an independent experiment. A monolithic workflow may rerun everything or, worse, regenerate a manuscript from scratch and introduce new inconsistencies. A dependency-aware workbench can invalidate only the affected branches. Validated outputs outside the dependency cone remain stable. The system becomes both more efficient and more auditable because correction is localized.

Software-like science also requires tests, but scientific tests are heterogeneous. Unit tests can verify functions, schema checks can enforce data structure, and theorem provers can establish formal properties. Statistical checks can test model assumptions or sensitivity. Laboratory controls can reveal contamination or instrument drift. Peer review can identify conceptual omissions. Replication can expose dependence on local conditions. The workbench must not collapse these into one green status indicator. A build can succeed while the scientific interpretation fails. An executable notebook can reproduce a biased estimator perfectly. A proof can be valid for a formal statement that does not represent the intended phenomenon. The system must show which dimensions passed and which remain open.

The deepest difference from software lies in relation to the world. Software correctness is ordinarily judged against a specification, even when the specification is incomplete or contested. Scientific inquiry often seeks the specification itself. The relevant variables, categories, mechanisms, and evaluators may be unknown at the beginning. This is why executable science cannot be a rigid compilation pipeline from complete requirements to final truth. It needs living specifications that can change under evidence, branches that preserve competing frames, and explicit records of semantic revision. The repository is not only a place where answers are built. It is where the language of the question is revised.

Table 3. The executable research object

Component	Operational role
Research specification	Records objectives, admissible claims, assumptions, invariants, methods, evidence thresholds, budgets, stopping rules, risks, and decision rights.

Component	Operational role
Claim and dependency graph	Connects each assertion to the data, code, models, protocols, transformations, and prior claims on which it depends.
Execution graph	Represents pipelines, agentic branches, tool calls, environments, validators, and promoted artefacts as rerunnable work rather than narrative reconstruction.
Evidence and criticism layer	Attaches supporting results, null findings, counterexamples, reviews, disagreements, uncertainty, and the status of unresolved objections.
Versioned rationale	Preserves consequential changes in interpretation, evaluator choice, scope, and release status without requiring indiscriminate retention of every private thought.
Public views	Produces papers, reports, dashboards, datasets, and interfaces as signed projections of a larger governed object.

2.2 Version control for claims, evidence, and reasons

A scientific version-control system must extend beyond file history. It needs a model of what changes mean. The core entities are claims, evidence, artefacts, specifications, decisions, and authorities. Each has an identity and a lifecycle. Their relations form a graph rather than a folder tree.

A claim is an assertion whose type determines its obligations. It may be a formal identity, an empirical regularity, a causal mechanism, a comparative advantage, a scaling law, a robustness statement, a novelty claim, a safety claim, or an interpretive conclusion. The type matters because evidence does not transfer automatically. A repeated correlation can support an empirical regularity without establishing mechanism. A benchmark improvement can support comparative performance under one budget without establishing general superiority. A formal proof can support an identity without showing that the formalization captures the intended system.

Evidence is not a generic confidence score. It is a typed relation between an artefact or observation and a claim. An executed test bears on implementation fidelity. A proof object bears on formal validity. A randomized experiment may bear on a causal effect under assumptions. A negative control may weaken a mechanism. An external replication bears on portability. A specialist review bears on a bounded judgment whose scope must be recorded. The same evidence may support one claim and undermine another. A result showing that a system works can simultaneously weaken the proposed explanation if the effect survives removal of the supposedly essential component.

Artefacts are the material objects through which research operates: datasets, code, environments, protocols, models, prompts, instrument outputs, notebooks, figures, annotations, and reports. They require content identity, versioning, and provenance. A result must be linked to the exact code and data states that produced it, not merely to a repository name. Dynamic dependencies such as external APIs, model versions, retrieval indexes, and policy configurations must be captured where they can affect the outcome. In AI-assisted research, prompts and model settings may matter, but preserving them is insufficient. The system must also record tool outputs, selected sources, execution environments, and the policy that determined which intermediate outputs entered persistent memory.

Specifications state what the inquiry is trying to accomplish and how it is governed. They contain objectives, constraints, evidence thresholds, budgets, stop rules, and decision rights. A claim may

be syntactically valid yet out of scope because it no longer answers the specification. A branch may produce an interesting result but require a new research-object version if the objective changes. This distinction prevents semantic drift from being disguised as progress.

Decisions connect evidence to status. A claim may be proposed, specified, implemented, reproduced, challenged, narrowed, promoted, held, rejected, superseded, or released. Status changes should not occur automatically from a model's confidence. They require a rule and, at higher levels, an authority. The decision record should state which evidence was considered, which objections remain, and why the transition was authorized. This creates a history of reasons.

The history of reasons differs from a full conversational log. Conversations contain repetition, private speculation, irrelevant context, and sensitive information. They may also contain hidden chain-of-thought material that is neither reliable evidence nor appropriate for disclosure. What matters for scientific continuity is the set of consequential transitions. A claim changed because a negative control succeeded. A branch stopped because the expected information gain no longer justified the cost. A dataset was excluded because consent did not cover the proposed use. A reviewer changed a verdict after an independent reproduction. An institution delayed release because the evidence threshold for the intended consequence had not been reached.

This history enables scientific operations analogous to diff, blame, merge, and revert, but with more careful semantics. An epistemic diff shows which claims, assumptions, and evidence relations changed between versions. Attribution identifies the people and systems responsible for artefacts and judgments without pretending that authorship is a single scalar. A merge combines compatible branches while preserving unresolved conflict. A revert restores a previous operational state but does not erase the fact that later evidence existed. A fork allows a community to pursue a different interpretation or policy without corrupting the history of the original object.

Conflict must be first-class. Conventional documents often resolve disagreement by producing one accepted text. A research object should allow multiple live interpretations of the same evidence. Two teams may agree on data and analysis but disagree about mechanism or significance. A regulator may accept a result for exploratory use while a clinical body rejects deployment. A statistician may judge the estimator valid under assumptions that a domain expert considers implausible. The graph should preserve these views and the authorities that endorse them. Consensus becomes an outcome, not an enforced schema.

Version control also supports negative results. Today, failed experiments and abandoned branches are often inaccessible, making future researchers repeat avoidable work and training AI systems on compressed success narratives. In a workbench, a negative result can close a branch, constrain a hypothesis, or update the expected value of a method. It should be preserved with enough context to be reusable: the specification it tested, the environment, the evidence, the reason for stopping, and the boundary beyond which the result should not be generalized. Negative evidence is not merely archival. It is executable memory.

Provenance must be proportional to consequence. Recording every intermediate token and transient state is expensive, invasive, and often useless. A low-risk exploratory calculation may require only code, data, environment, and a brief rationale. A clinical or safety-critical result may require instrument calibration, data lineage, model and policy versions, access authority, independent witness, and release approval. The research object should declare a provenance profile appropriate to the domain and intended use.

Cryptographic commitments and transparency services can strengthen integrity and ordering, but they do not prove scientific truth. A hash can show that an artefact changed. A signature can show that a key authorized a statement. An append-only log can make retrospective rewriting detectable. None establishes that an instrument was calibrated, a participant consented freely, a

model was unbiased, or a claim is correct. The infrastructure should decompose trust into inspectable evidence rather than replace institutional trust with technological rhetoric.

Long-lived verifiability introduces maintenance obligations. Code environments decay, container registries disappear, model APIs change, cryptographic algorithms age, and external data sources are revised. A research object must preserve not only content but proof liveness: enough information and executable infrastructure to interpret historical evidence later. Migration should itself produce evidence linking old and new states. Otherwise, the repository may contain immutable commitments to artefacts that no one can reconstruct.

A useful way to make that semantic layer concrete is to define the scientific equivalent of a commit. A research commit should not mean merely that files were saved. It should identify the prior accepted state, the changed artefacts, the affected claims, the specification clause that motivated the change, the validators that ran, the evidence produced, and the authority under which the transition was admitted. The commit message becomes a compact rationale, while the graph contains the inspectable support. Routine low-risk changes may be admitted automatically when deterministic checks pass. Changes that narrow a central claim, alter an evaluator, replace a dataset, or raise the consequence of release should require explicit review. In this model, continuous integration becomes continuous epistemic integration: every material change triggers the smallest competent set of tests, reproductions, provenance checks, and policy gates.

Scientific branching also needs stronger semantics than software feature branches. A branch may encode a rival mechanism, an alternative causal model, a different preprocessing decision, an adversarial interpretation, or a distinct ethical and regulatory position. Its purpose should be declared before results are known, otherwise branching becomes a retrospective device for hiding researcher degrees of freedom. A merge should occur only when the branches are compatible at the level of assumptions and evidence. When they are not, the repository should preserve a structured conflict. A conflict is not a technical failure to be eliminated by fluent synthesis. It may represent the most informative state of the inquiry: two interpretations that share observations but differ in causal commitments, or two release policies that assign different weight to risk.

Tags and releases acquire epistemic meaning as well. A tag may identify a preregistered analysis state, a frozen dataset, an externally reproduced result, a regulatory submission, or the exact object from which a paper was rendered. A release is therefore not only a bundle of artefacts. It is a policy assertion that a defined set of claims has reached a declared evidence level for a declared audience and use. Later evidence may supersede that release without erasing it. This matters because scientific correction is not equivalent to rewriting a web page. People and institutions may have acted on an earlier state. A responsible system must preserve which evidence and authority were available at the time.

The issue tracker also changes role. Open questions, suspected confounds, missing controls, replication failures, reviewer objections, and safety concerns become addressable objects linked to the relevant claims and versions. Closing an issue requires evidence or a reasoned decision, not merely a conversational assurance. Some issues remain intentionally open at release because uncertainty is irreducible or because the required experiment is not yet feasible. Their visibility prevents a polished manuscript from converting absence of resolution into appearance of consensus.

Such repositories would support new forms of scientific review. A reviewer could request a semantic diff between the submitted claim and its preregistered ancestor, inspect whether a positive result survived evaluator changes, or run the dependency slice affected by a disputed assumption. A replication team could fork the object while retaining ancestry, attach independent evidence, and propose a merge or a persistent divergence. Meta-researchers could study how claims mature, which kinds of criticism cause revision, and where evidence repeatedly fails to propagate into public

conclusions. These capabilities are plausible with extensions of existing version control, provenance models, research-object packaging, workflow systems, and policy engines. The difficult work is not storing more files. It is agreeing on the minimal identities, relations, and transition rules that make the stored history scientifically meaningful.

The Git analogy is therefore powerful but incomplete. Git answers whether bytes changed and how branches relate. Executable science must also answer whether the meaning of a claim changed, whether the evidence remains valid, whether the authority is current, and whether the object can still be interpreted. The future scientific repository will need a semantic layer above ordinary version control, not a replacement for version control.

2.3 The paper as a signed view: transparency, privacy, and public argument

If the research object becomes the durable system of record, the scientific paper does not disappear. Its role becomes clearer. The paper is a selected, signed, public argument. It explains why a question matters, which evidence is decisive, how alternative interpretations were treated, and what the authors are prepared to defend. It is a view over the research object, not an automatic dump of the object's contents.

This distinction could improve scientific writing. Current papers often combine narrative, implementation detail, reporting compliance, and repository pointers in a format that satisfies none of these purposes completely. A view-based architecture allows the public article to remain conceptually coherent while the underlying object provides executable depth. The paper can state the central claims and the evidence that changed the authors' position. A reviewer can navigate from each claim to methods, code, data, negative controls, and prior criticism. A replicator can obtain a frozen environment. A policymaker can inspect consequence-specific evidence and dissent. The manuscript can become shorter without becoming less transparent because compression is no longer equivalent to deletion.

The signature is important. Authorship in this model is not a claim that the authors manually produced every sentence, line of code, or experiment. It is an acceptance of responsibility for the public argument and its release state. The author or institution declares that the specification is represented honestly, the cited evidence exists, known material limitations are disclosed, and the claimed scope is justified. AI use should be reported at the level necessary to assess provenance and common-mode risk, but disclosure should not substitute for responsibility. "The model produced it" is not an authorship category and cannot function as a liability shield.

The research object can support richer credit than a conventional author list. Contribution records can identify hypothesis design, software, data curation, instrument operation, formal verification, causal analysis, replication, review, governance, and maintenance. Machine contributions can be recorded as tools and activities rather than fictional persons. Human contributors can receive credit for closing expensive branches, constructing decisive controls, maintaining benchmarks, or curating evidence even when those actions do not produce visible prose. This can align recognition more closely with the work that makes knowledge cumulative.

Transparency, however, must be layered. A naïve response to automated science is to demand complete logging. Such a regime would be both technically excessive and intellectually harmful. Research requires a protected interval of uncertainty. Scientists need to explore interpretations that may be wrong, discuss sensitive possibilities, and revise ideas without every provisional statement becoming a permanent performance record. Full logging can expose personal data, confidential collaborations, unpublished intellectual property, security-sensitive methods, and the private context of reviewers or participants.

A defensible architecture separates at least four visibility regimes. Private exploration allows uncommitted work and ephemeral interaction. Collaborative state contains artefacts and decisions shared within the research team. Institutional audit preserves evidence needed for integrity, safety, and accountability under controlled access. Public release exposes the claims, supporting evidence, provenance, and unresolved dissent appropriate to external reliance. The boundaries are not fixed universally; they depend on domain, law, consent, and consequence. What matters is that the transition between regimes is explicit and authorized.

The right to be wrong before publication should be treated as an epistemic right. It is not a right to conceal fraud or suppress known harm. It is the protection of provisional thought from premature institutionalization. Without it, researchers and agents will optimize for records that look defensible, avoiding risky questions and candid disagreement. The workbench should preserve committed reasons, not every cognitive trace.

At the same time, public claims need stronger traceability than ordinary papers provide. A released view should identify the exact research-object version it represents. If data, code, or interpretation later change, the public record should show whether the article remains valid, has been superseded, or now carries a warning. Retraction should not make history disappear. Correction should be an explicit successor state linked to the prior one. External replication should attach as new evidence rather than remain a disconnected citation.

This model also changes peer review. Review becomes an intervention on the research object, not merely comments on a PDF. A reviewer can challenge a specific claim, request a branch, attach an executable test, identify a missing source, or restrict the scope of a verdict. The authors' response can link to the resulting artefact and show how claim status changed. Post-publication criticism can continue in the same structure, allowing the object to accumulate evidence rather than forcing every dispute into a new article.

There is a danger that such infrastructure becomes bureaucratic. If every exploratory project requires a large ontology, complex access policy, and formal approval for minor changes, the workbench will be rejected or will encourage performative compliance. The minimum viable research object should be small. It needs stable identity, versioned specification, claims, evidence links, provenance sufficient for consequence, and a record of decisions. Additional structure should be introduced only where it improves control, reproducibility, or judgment. The architecture must support progressive formalization: ordinary prose and Markdown at the surface, typed representations where useful, formal proofs or deterministic validators where available.

This progressive approach is essential for adoption. Scientists already use repositories, notebooks, workflow engines, electronic laboratory notebooks, preregistrations, and data packages. The transition to executable science should connect these tools rather than demand one universal platform. Interoperable profiles and open formats matter more than a proprietary “AI scientist” interface. A research object should survive changes in model vendor, orchestration framework, storage service, and institutional policy.

The likely end state is not a single global GitHub for science. Scientific domains differ in privacy, data scale, instrument control, intellectual property, and governance. The more plausible architecture is federated. Research objects retain stable identities and portable evidence profiles while institutions operate their own storage, access, and validation services. Public repositories expose release views and reproducibility packages. Sensitive domains provide verifiable summaries, controlled execution, or independent attestations without publishing raw data. Interoperability occurs at the level of claims, provenance, evidence, and version relations.

The normative conclusion is that the scientific article should remain the place where humans make an intelligible public argument. It should no longer be forced to act as the entire memory of

the inquiry. The repository preserves execution and change; the paper preserves significance and responsibility. Their separation is not a demotion of writing. It is a recognition that, in an age of generative abundance, rhetoric must be connected to a larger object capable of resisting it.

Chapter 3. Specification-Driven Inquiry

3.1 The living research specification, the Specification Ladder, and the Mirroring Pattern

A prompt is an interaction. A research specification is a persistent commitment. The distinction is architectural rather than linguistic. A prompt may contain detailed instructions, and a specification may be written in prose. What makes the specification different is that it has an identity, a version, a declared scope, and relations to downstream artefacts. It can be reviewed before execution, compared with later states, and used to determine whether a result still answers the original question.

This distinction matters because language models are unusually capable translators. They can turn a broad ambition into requirements, requirements into plans, plans into code, code into explanations, and evidence into prose. The same capability that makes them useful also makes semantic drift easy. Every translation can omit a constraint, strengthen an interpretation, resolve an ambiguity prematurely, or substitute a convenient proxy for the intended objective. A monolithic prompt hides these transformations inside one generation. A specification-driven system externalizes them as reviewable artefacts.

A mature research specification should state the purpose of the inquiry, the kinds of claims that may be advanced, the assumptions and invariants that currently hold, the admissible methods, the evidence thresholds, the resource budget, the stop conditions, the relevant risks, and the decision rights. It should distinguish objectives from preferred mechanisms. “Determine whether intervention A changes outcome Y under population and resource constraints Z” is an objective. “Use model family M” is a methodological preference that may need to be revised. Conflating the two allows a favoured technique to redefine the question it was meant to answer.

The specification cannot be complete at the beginning. Research differs from ordinary contract implementation because the relevant vocabulary, representation, or decisive experiment may be unknown. The specification must therefore combine commitments with declared freedom. It should identify what must remain invariant, what may be explored, which uncertainties are recognized, and which transformations require human approval. This makes it a living constitutional object rather than a frozen requirements document.

The Specification Ladder organizes this living object across levels of abstraction. At the top are mission and consequence: why the inquiry matters, to whom, and under which risk regime. The next level defines the research question, claim types, and scope. Below it lie competing hypotheses and representations, followed by estimands, protocols, datasets, models, experiment plans, software modules, instrument procedures, analyses, and release claims. Each level is more concrete than the previous one, but each remains linked to the intention it interprets.

The ladder provides control because ambiguity can be resolved at the level where it belongs. A disagreement about social purpose should not be hidden inside a statistical parameter. A dispute about a causal estimand should not be postponed until manuscript review. A conflict about an instrument’s operational definition should not appear only as unexplained variance in the final data. By materializing intermediate commitments, the system creates earlier points of correction.

The Mirroring Pattern extends this idea. The structure of the specification should mirror the structure of the artefact or programme being produced closely enough that local intent can guide local generation and validation. In software, a high-level product vision is refined into functional and non-functional requirements, architecture, modules, interfaces, files, tests, and code. In research, a

mission is refined into questions, claims, designs, experiments, analyses, evidence objects, and publication views. The final artefact is no longer the sole location of intent. It is the most concrete layer in a chain.

Mirroring does not require one-to-one bureaucratic decomposition. Some scientific objects are cross-cutting. A safety constraint may apply to several experiments, and one dataset may bear on multiple claims. The point is that the workbench should maintain explicit correspondence between levels. A downstream artefact should reveal which specification clauses it implements and which evidence obligations it serves. When an upstream commitment changes, the system can identify the affected subgraph.

This structure is especially useful for long-lived research. Projects outlast particular conversations, models, and team members. Initial intentions are lost, code becomes the de facto specification, and later modifications reconstruct rationale from scattered notes. A persistent ladder preserves the project as an evolving chain of artefacts. New evidence can trigger localized revision instead of global reinterpretation. High-level intent remains relatively stable and global; low-level specifications remain local and replaceable; execution contexts can be reconstructed as needed.

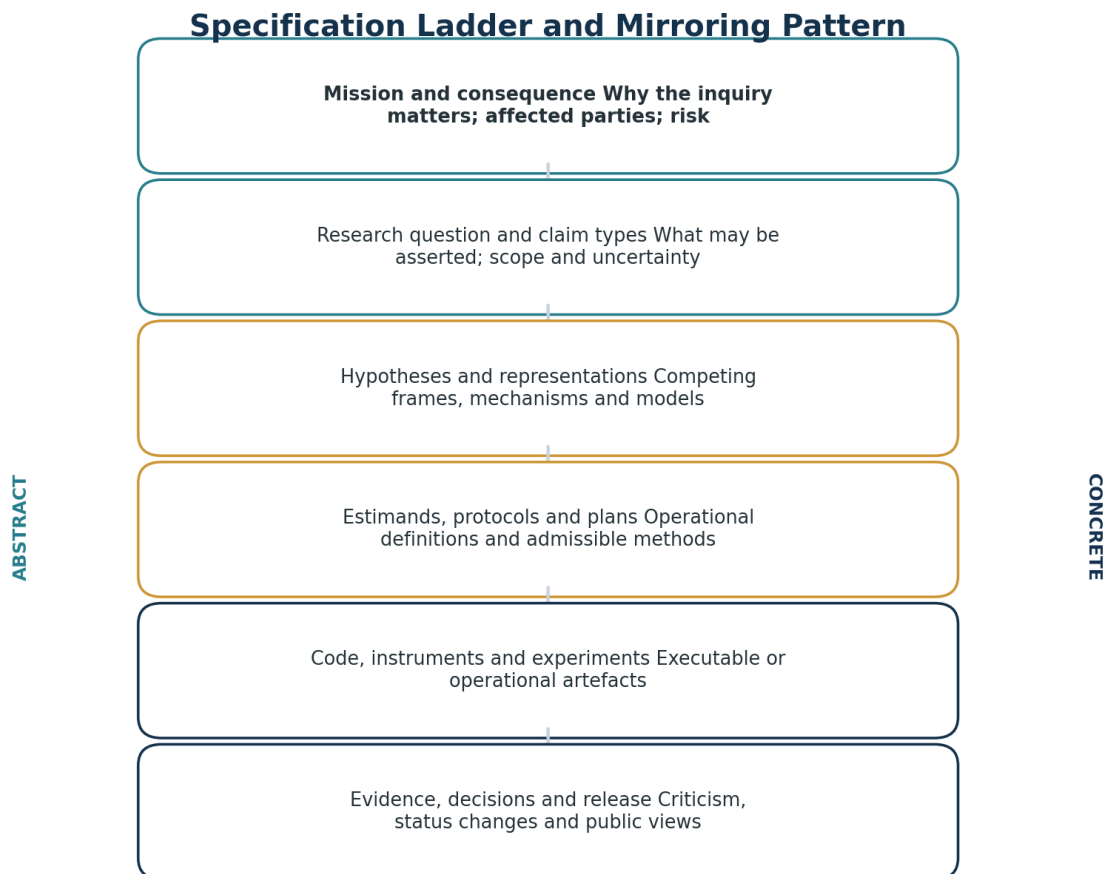
Controlled Natural Language can provide a practical authoring surface. Fully formal specifications are expensive and often impossible when concepts remain open. Unrestricted prose is readable but difficult to validate. A controlled language constrains syntax and vocabulary enough to make claims, dependencies, modalities, and exceptions extractable while remaining close to natural scientific expression. Existing work in requirements engineering, including FRET, demonstrates that controlled language can connect readable requirements to formal semantics and verification artefacts (Giannakopoulou et al., 2020). Language-model programming systems such as LMQL and DSPy demonstrate complementary ways to make generation and model calls programmable and optimizable (Beurer-Kellner et al., 2023; Khattab et al., 2023).

The strongest architecture uses two representations. A human-facing projection remains readable, reviewable, and diffable. A typed intermediate representation preserves claim identities, roles, dependencies, constraints, provenance, admissibility conditions, and validation hooks. The projection is not a loose summary; it is generated from or synchronized with the intermediate representation. The system can ask a model to interpret an ambiguous clause, but it should record the interpretation as a proposed transformation rather than silently changing the IR.

SOP Lang, in the ACHILLES research direction, is intended to serve this role. SOP Lang Plan expresses stepwise coordination for interactive work. SOP Lang Pipeline expresses build-like dependency graphs with promoted artefacts and selective recomputation. These constructs should be understood as an evolving research substrate. Their value will depend on whether they can preserve semantic commitments across real tasks more effectively than ordinary documents and agent transcripts.

The Specification Ladder also determines where human review is most valuable. Human-in-the-loop should not mean that a person clicks approve after every tool call or signs only at the end. Review should occur at transformations where error would propagate widely or where legitimacy requires judgment. A scientist may approve the question and admissible claim types, a statistician the estimand and identification assumptions, a laboratory lead the protocol and safety envelope, and an editor the public release. Lower-risk local steps can execute automatically under the approved specification.

The ladder is therefore not simply documentation. It is a control surface, a dependency model, and a governance map. It tells the system what may be automated, which constraints must survive, what evidence is required, and who can change the rules. It converts the project from a sequence of prompts into a maintainable programme of inquiry.



Mirroring links every concrete artefact to the specification clause it interprets.

Figure 3. The Specification Ladder and Mirroring Pattern. Each more concrete artefact remains linked to the level of intent it interprets.

3.2 Specification explosion and portfolio science

A common mistake in specification-driven development is to imagine that a better specification should produce one better plan. That intuition is appropriate when the task is implementation under stable requirements. It is insufficient for research, where the specification defines a space of uncertainty. A scientific specification should often compile into a portfolio of competing branches rather than a single trajectory.

This controlled expansion is specification explosion. A research question is transformed into alternative hypotheses, representations, datasets, experimental designs, baselines, negative controls, scaling tests, sensitivity analyses, and attempts at refutation. Each branch records which clause motivated it, what resources it may consume, what evidence it can produce, and what condition will stop it. The purpose is not to maximize the number of branches. It is to prevent the first plausible framing from becoming the whole research programme by default.

Portfolio science has several advantages. It separates the objective from any one mechanism. It makes alternative explanations visible before a result is interpreted. It allows constructive and adversarial work to proceed in parallel. It supports resource allocation based on expected information gain rather than narrative momentum. It preserves negative results as constraints on the remaining space. It also reduces the risk that a language model's initial completion, which is strongly shaped by familiar patterns, determines the entire direction of inquiry.

Branches should be heterogeneous in function. Constructive branches attempt to make a hypothesis work. Adversarial branches search for counterexamples, confounds, leakage, simpler explanations, and failure regimes. Baseline branches implement the strongest relevant alternatives under matched resources. Translation branches test whether the claim survives a different representation. Scaling branches examine whether a local effect persists under realistic load, noise, data size, or physical constraints. Replication branches reproduce the result with independent code, tools, data, or personnel. Governance branches test whether the proposed use satisfies consent, safety, and release obligations.

The specification should make these branch types explicit because a portfolio containing only constructive variations is not pluralistic. Ten agents optimizing the same mechanism under the same evaluator create depth of search, not independence. Diversity must be measured in dependencies and assumptions. A useful portfolio varies model families, evidence sources, evaluators, and explanatory frames where the cost is justified.

The process resembles search over a design space, but the scientific objective is richer than optimization. A candidate can achieve a high score while weakening the proposed explanation. A failed branch can be valuable because it rules out a mechanism. A simpler model can reduce novelty while increasing understanding. The state of the portfolio therefore cannot be summarized by one scalar fitness value. It needs a graph of claims, evidence, and unresolved alternatives.

The experiment graph is the operational form of this portfolio. Nodes represent questions, hypotheses, methods, experiments, analyses, critics, and decisions. Edges encode dependency, support, contradiction, refinement, or supersession. A node has a cost estimate, an expected evidence type, a risk profile, and a stopping condition. Results update the graph rather than merely producing prose. The next action can be selected by heuristics such as expected information gain, uncertainty reduction, coverage of the hypothesis space, cost, risk, or the probability of a decisive counterexample.

Bayesian optimization and active learning already operationalize related ideas in bounded settings. They choose experiments that are expected to improve an objective or reduce uncertainty efficiently (Settles, 2009; Frazier, 2018). Optimal experimental design formalizes choices about informative measurements. Specification explosion generalizes the idea beyond parameter optimization. The branches may change the representation, causal model, evaluator, or question itself. Some evidence updates probabilities; other evidence changes the space in which probabilities are meaningful.

The main difficulty is combinatorial growth. Uncontrolled explosion consumes compute, instruments, and attention without increasing knowledge. A serious implementation needs branch governance. Each branch receives a budget, a stop rule, and a declared value of information. Duplicate branches can be clustered by dependency fingerprints. Low-value branches can be pruned after inexpensive checks. Promising branches can be escalated gradually. The portfolio should maintain diversity without treating every generated variation as worthy of execution.

A useful policy is progressive evidential investment. At the first stage, branches are cheap specifications or simulations. They are tested for internal coherence, literature collision, feasibility, and obvious counterexamples. A smaller set advances to executable prototypes. Fewer still receive expensive computation, laboratory resources, specialist review, or external replication. Promotion requires new kinds of evidence rather than greater confidence from the same generator.

This policy produces a scientific funnel, but unlike a business funnel, rejected branches remain informative. Their stopping reasons update the specification and constrain future search. If a branch fails because the effect disappears under matched compute, the system should record that dependency. If it fails because the operational definition was invalid, the workbench should update

the relevant specification level. If it fails because the required data are ethically inaccessible, the governance constraint becomes part of the research object.

Specification explosion also supports model pluralism. Different representations reveal different properties and failure modes. A mechanistic model may support explanation but fit poorly. A black-box predictor may forecast well but fail under intervention. A symbolic approximation may allow proof. A simulation may capture interactions at high cost. A qualitative framework may identify categories omitted by quantitative data. The workbench should not force these into one ontology prematurely. It should represent translations and contradictions explicitly.

The role of the LLM is particularly strong at the beginning of the explosion. It can propose distant analogies, translate terminology across fields, enumerate assumptions, and generate candidate controls. Yet it should not decide by itself which branches deserve expensive escalation. Ranking by a model can be useful as one input, but correlated self-evaluation risks reinforcing the generator's biases. Programmatic checks, retrieval, independent models, domain heuristics, and human review should jointly shape the portfolio.

This is one reason the specification must contain decision rights. A system may be authorized to generate and prune low-risk computational branches automatically. It may need approval before accessing sensitive data, running costly experiments, operating physical equipment, or releasing a dual-use result. Autonomy becomes a property of a branch under a contract, not an all-or-nothing property of the “AI scientist.”

The likely scientific benefit of specification explosion is not that it discovers a larger number of positive results. It is that it makes premature convergence harder. A research programme can explore multiple frames while preserving common objectives and comparable evidence. It can reward the branch that produces the most decisive criticism, not only the branch that produces the most impressive metric. In a world where generative systems can create alternatives cheaply, disciplined plurality becomes an engineering requirement.

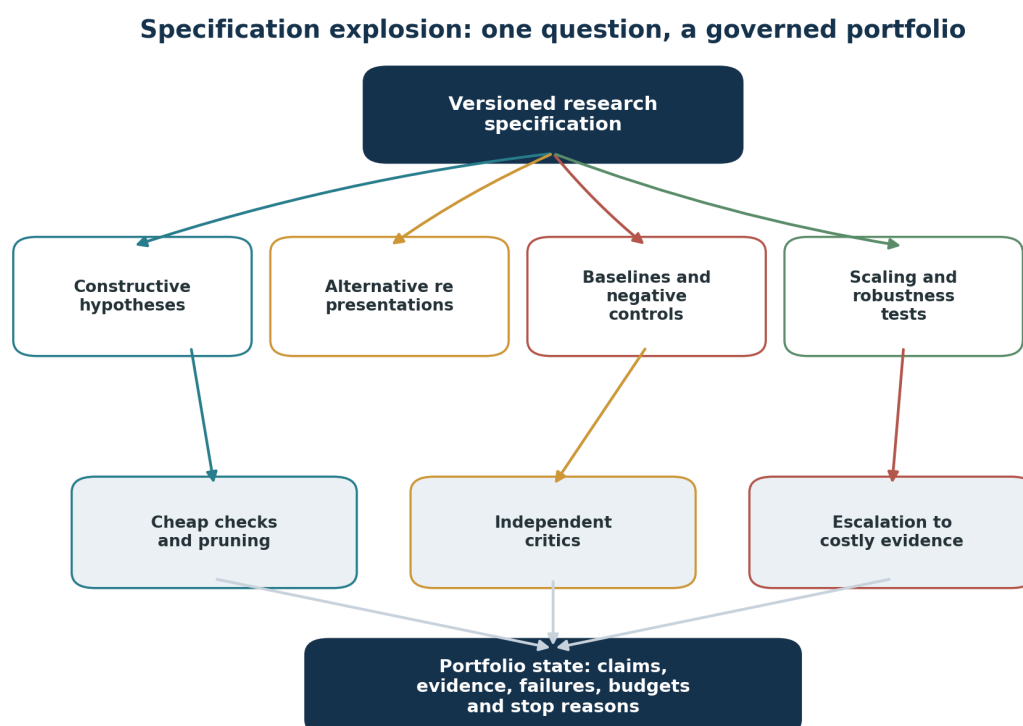


Figure 4. Specification explosion. A persistent specification expands into a governed portfolio rather than a single linear plan.

3.3 Hybrid execution: deterministic pipelines, bounded agents, and selective recomputation

Two paradigms dominate contemporary discussions of AI-assisted work. The pipeline paradigm defines a sequence or graph of operations in advance. The agentic paradigm allows a model to choose actions dynamically in response to intermediate state. They are often treated as competitors. In executable science, they are complementary layers.

Pipelines are strongest when dependencies, transformations, and evaluators are known. Data ingestion, schema validation, environment construction, code execution, proof checking, statistical reporting, provenance capture, and artifact packaging should be deterministic where possible. Determinism reduces cost, variance, and audit difficulty. A stable operation should not be repeatedly reinterpreted by a language model merely because a model can perform it.

Agents are strongest where the next useful action cannot be fully enumerated. Literature exploration, hypothesis generation, representation choice, diagnosis of unexpected results, design of counterexamples, and revision of an experiment plan may require adaptive reasoning. ReAct, Reflexion, tree-search methods, and multi-agent systems show how models can interleave reasoning, tool use, feedback, and self-correction (Yao et al., 2023; Shinn et al., 2023; Madaan et al., 2023). Their flexibility is valuable, but a transcript is a weak long-term state representation. It contains the history of interaction without necessarily preserving the logical structure of the project.

The most credible architecture places bounded agentic loops inside an explicit outer graph. The pipeline specifies inputs, outputs, permissions, budgets, validators, and promotion rules. A node may invoke a deterministic function, a theorem prover, a database query, a human review, or an agent permitted to explore locally. The agent returns named artefacts and evidence, not merely a conversational answer. The outer runtime decides how those outputs enter persistent state.

This architecture can be described as deterministic exterior, adaptive interior. It avoids the rigidity of a fully predefined workflow while preventing an agent's local improvisation from becoming the project's implicit constitution. The agent may propose a new branch or request a change to the specification, but the change is committed explicitly. It may repair code within a sandbox, but promotion requires tests and provenance. It may summarize evidence, but the underlying sources and claim links remain outside the summary.

The distinction between output and memory is essential. An agent's intermediate statement should not automatically become reusable project knowledge. Memory promotion requires a policy. A result may remain provisional until validated, be scoped to one branch, or be rejected after criticism. Without this separation, an early hallucination or mistaken interpretation can contaminate later context and create self-confirming loops. A research workbench should treat memory as a governed database of artefacts and commitments, not as accumulated conversation.

Selective recomputation follows from the graph structure. When an upstream specification, dataset, model, or policy changes, the runtime identifies affected nodes. Deterministic nodes rerun automatically. Agentic nodes receive a bounded frame containing the relevant change rather than the entire project history. Human review is requested only where meaning or authority changes. This approach improves efficiency and reduces the risk that global regeneration introduces unrelated variation.

Context engineering becomes a compilation problem. Instead of inserting the whole transcript into every prompt, the runtime projects the smallest relevant slice of specification, artefacts, evidence, and unresolved questions for the current node. High-level intent remains available through stable references. Local detail is loaded on demand. This reduces token cost, but its more important function is epistemic: it makes the basis of a local judgment inspectable.

Stopping is part of execution semantics. Agentic systems often continue until a budget or heuristic completion criterion is reached. Research needs richer stop rules. A branch may stop because the claim was refuted, the expected information gain became too low, the resource budget was exhausted, a safety threshold was reached, an unresolved dependency requires a human, or the result is sufficient for the intended consequence. The stop reason becomes an artefact. Abandonment is not failure of the system; it is a scientific outcome.

This hybrid model also clarifies evaluation. A pipeline node can be tested against its contract. An agentic node can be evaluated over trajectories, cost, state integrity, and the quality of promoted artefacts rather than the eloquence of its final answer. Regression suites can include seeded confounds, adversarial sources, missing data, instrument failures, and specification changes. Agent evaluation becomes closer to software testing without assuming that open-ended scientific judgment has one deterministic oracle.

Industry is already moving toward this combination. Specification-driven coding environments turn natural-language goals into requirements, designs, and task plans before agents implement code. Agent frameworks provide graphs, routers, state stores, tool protocols, and human checkpoints. Model Context Protocol standardizes access to tools and data under host-mediated security boundaries. Structured-output libraries constrain generation through schemas and grammars. These are important signals, but they remain mostly application infrastructure. They do not yet provide a general epistemic runtime for claims, evidence, critic independence, and decision rights across sciences.

The ACHILLES proposal extends the hybrid pattern. SOP Lang Pipeline supplies the dependency graph. SOP Lang Plan supports interactive coordination. Generalized skills or interpreters implement local regimes. MRP-VM is conceived as the runtime that selects a regime, constructs a bounded frame, executes locally, demands evidence, and reintegrates the result into the shared intermediate representation. The proposal is ambitious and remains to be validated. Its scientific value will depend on whether it can outperform simpler workflows on trace fidelity, validation quality, selective recomputation, and cost.

The design principle is nevertheless robust independent of one implementation. Use language models where interpretation and proposal are valuable. Use symbolic, statistical, and deterministic methods where they provide stronger or cheaper guarantees. Encapsulate adaptive loops inside explicit contracts. Preserve outputs as versioned artefacts. Promote memory through policy. Treat stopping and revision as first-class operations. The future scientific agent is not a chatbot with more tools. It is a local source of adaptive work inside a governed system of inquiry.

Chapter 4. The Architecture of the Scientific Workbench

4.1 Local autonomy, global evidence

The missing instrument of automated research is not another universal agent. It is a scientific workbench: an operating layer that connects intent, execution, evidence, criticism, and release over the lifetime of an inquiry. The workbench resembles an integrated development environment only in its architectural role. An IDE connects source code to build systems, tests, debuggers, version control, and collaboration. A scientific workbench connects specifications to research branches, domain tools, experiments, claim-evidence graphs, review, and accountable decisions.

The governing principle is local autonomy with global evidence. Local components may act adaptively within bounded responsibilities. A literature agent can explore terminology and sources. A coding agent can implement a method in a sandbox. A theorem prover can certify a formal statement. A laboratory controller can optimize an experiment under an approved safety envelope. A critic can search for a counterexample. None of these components owns the truth state of the project. Their outputs enter a shared evidence structure where provenance, dependencies, status, and authority remain explicit.

A reference architecture begins with intent and governance. The workbench records the research objective, consequence class, permissible data, resource limits, and authorities. This layer determines which actions can execute automatically and which require approval. It also defines the release regime. Exploratory computational work can be relatively permissive; physical experiments, personal data, dual-use procedures, and public recommendations require stronger controls.

The specification compiler transforms intent into a graph of branches and obligations. It does not produce a complete plan once and for all. It creates a current execution graph, identifies unresolved ambiguities, and marks places where alternative representations or methods are admissible. Every node declares inputs, outputs, preconditions, side effects, expected evidence, cost, and stop conditions. Nodes can be deterministic, agentic, symbolic, human, or composite.

Execution occurs in controlled environments. Code and models should run in isolated, reproducible sandboxes where commands, dependencies, resource use, and outputs can be captured. Laboratory actions should be mediated by instrument interfaces, safety interlocks, calibration state, and recovery policies. Retrieval should record queries, source versions, access time, and selection policy. Model calls should record enough configuration to assess common-mode dependence without assuming that a private chain of thought is evidence.

The evidence layer is the workbench's centre. Outputs do not move directly to prose. They become artefacts linked to claims and specification clauses. A claim can have multiple supporting and opposing evidence relations. Critics attach typed verdicts. Status changes are explicit. Negative results and failed branches are retained when they constrain future work. The graph distinguishes an executed artifact from an independently reproduced artifact, a formal result from its empirical relevance, and a statistical association from a causal interpretation.

The adjudication layer applies promotion rules. Early claims can advance automatically through cheap checks. Later stages require independent reproduction, adversarial tests, specialist judgment, or external validation depending on domain and consequence. The adjudicator may itself be a policy engine for low-risk transitions, but high-stakes promotion requires named authority. The workbench records the decision and its reasons.

Finally, release generates views. A manuscript presents an argument. A reproducibility package presents executable dependencies. A reviewer view exposes unresolved obligations. A governance view presents authority, risk, and provenance. An agent-facing view condenses the current state into a bounded context. Views are projections, not independent copies. They point to a stable research-object version so that later changes remain traceable.

This architecture allows multiple autonomy patterns. A fixed computational workflow may execute end to end when the evaluator is strong. A multi-agent deliberation may generate and rank hypotheses while leaving experiments to humans. A robotic laboratory may close the loop for a narrow material-search problem. A theorem-search system may generate candidates indefinitely because invalid outputs are rejected by a proof checker. A clinical research workbench may automate literature, protocol checks, data analysis, and monitoring while reserving interventions and conclusions for governed teams. The architecture is common; the admissible autonomy differs.

The workbench must support failure as a normal state. Tools become unavailable, models produce malformed output, proofs fail, datasets change, agents exceed budgets, and reviewers disagree. Recovery should not depend on reconstructing a conversation. Nodes have idempotence and retry policies where appropriate. Partial artefacts are marked provisional. External side effects are transactional or compensatable. A failed instrument action should not be retried blindly. A research branch can pause with an unresolved obligation rather than forcing the agent to produce closure.

The workbench also needs observability. Operational monitoring records latency, cost, resource use, and failures. Epistemic monitoring records semantic drift, claim promotion, critic disagreement, evidence gaps, and unresolved dependencies. Governance monitoring records access, authority, and policy changes. These dimensions should not be collapsed. A fast, stable pipeline can still be epistemically weak. A rigorous branch can still violate consent. A compliant process can still ask an unimportant question.

The architecture's value can be tested empirically. It should reduce the time needed to localize errors, reproduce results, identify semantic drift, and assemble review packages. It should reduce common-mode critic failure and memory contamination. It should allow selective recomputation after an upstream change. It should make negative results reusable. If the additional structure does not improve these outcomes enough to justify its cost, it should be simplified.

This last condition matters because scientific workflow platforms often become heavy. The workbench should not require every laboratory to adopt one ontology or cloud service. A minimal implementation can begin with versioned Markdown specifications, ordinary repositories, structured claim identifiers, workflow metadata, and explicit review records. Formality should increase with consequence and with demonstrated benefit. The architecture is a set of invariant relations, not a demand for maximal infrastructure.

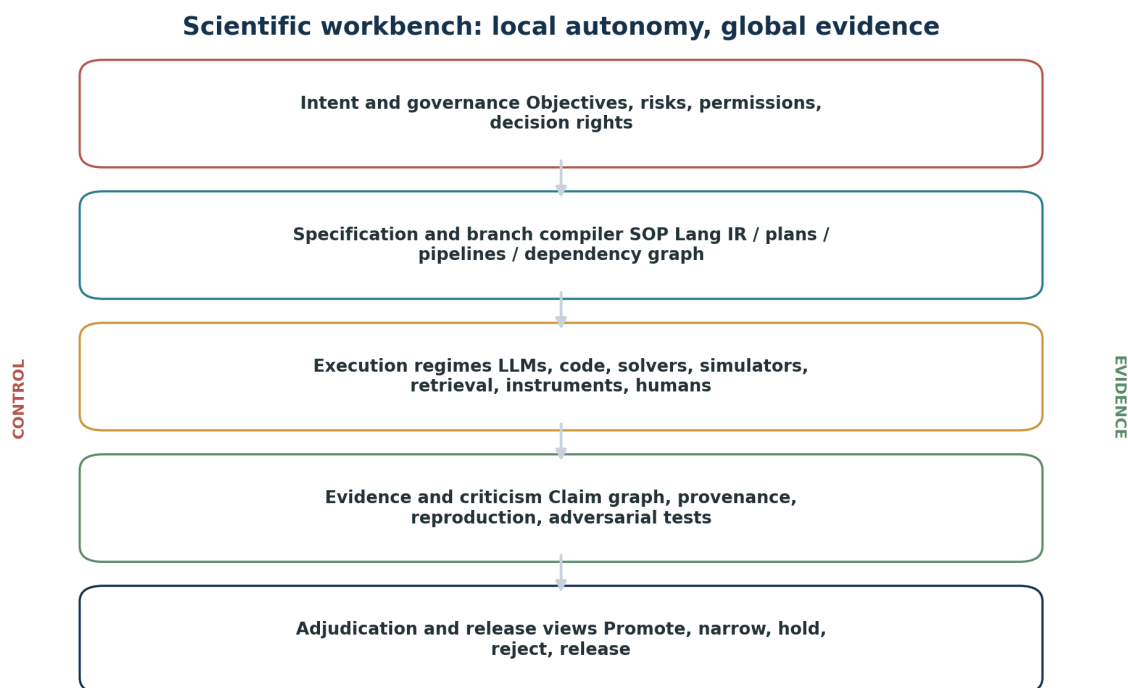


Figure 5. Reference architecture of the scientific workbench: local autonomy is bounded by global evidence and decision rights.

4.2 From industrial skills to scientific interpreters and MRP-VM

The modern agent ecosystem has converged on several practical abstractions. Tools expose atomic capabilities. Model Context Protocol and related interfaces standardize access to data and services while preserving host-mediated boundaries. Agent frameworks provide state graphs, routing, retries, and human checkpoints. Coding environments increasingly maintain project instructions, specialized agents, hooks, and specification artefacts. Anthropic’s Agent Skills popularize a package-like unit that combines instructions, resources, and repeatable workflows for a model. Kiro makes requirements, design, and task specifications first-class objects in an agentic development environment. These developments indicate a shift away from the monolithic prompt toward reusable, context-engineered execution units.

Their significance for science is substantial but limited. A conventional tool definition says what operation is available and what schema it accepts. A skill says more: when a procedure applies, how the model should use resources, and what workflow to follow. A spec-driven coding environment links intent to implementation and tests. An agent graph constrains control flow. Yet most industrial abstractions remain oriented toward task completion. They do not necessarily declare the epistemic scope of an output, the evidence required for acceptance, the independence of the critic, or the authority that can promote a scientific claim.

The generalization proposed here is to treat a skill as a bounded execution regime. A scientific interpreter does not merely perform a task. It declares the class of problems for which it is competent, the representations it expects, the assumptions it makes, the evidence it can produce, the validators it invokes, the failure states it recognizes, and how its output can be reintegrated. This regime may be implemented by an LLM, a subagent, deterministic code, a theorem prover, a statistical package, a causal-analysis engine, a simulator, a database query, a robotic procedure, a human review protocol, or a composition of these elements.

Calling these units “interpreters” emphasizes their role at the boundary between specification and execution. A research specification contains commitments at a relatively abstract level. The interpreter gives those commitments operational meaning in a local domain. A citation-integrity interpreter resolves claim references, retrieves sources, checks whether the cited passages support the claim, and returns evidence with uncertainty. A statistical-validity interpreter maps an estimand and data description to diagnostics, model fits, sensitivity analyses, and a scoped verdict. A theorem interpreter translates a formal claim into proof obligations and invokes a prover. A laboratory-safety interpreter checks a protocol against hazard rules and instrument constraints. A narrative LLM may participate, but it does not need to control every step.

A “super-skill” is a useful name for a higher-order regime that can select and compose local interpreters. It might receive a research claim, infer its type, construct the relevant validation plan, execute inexpensive checks, and route unresolved questions to specialists. In implementation, a super-skill may sometimes be a subagent with its own planning loop. In other cases, it should be a deterministic dispatcher or a neuro-symbolic program. The point is not to make every unit agentic. It is to package domain competence together with applicability and evidence obligations.

This distinction creates a path from today’s practical skills to a more rigorous scientific runtime. Existing skills can be used immediately as bounded instruction packages. Over time, their contracts can be enriched with typed inputs, outputs, provenance, preconditions, side-effect declarations, cost models, validation hooks, and evidence schemas. Where symbolic mechanisms become available, LLM judgment can be replaced. The model remains valuable as a proposer, translator, renderer, or bounded interpreter, while the runtime relocates validation into inspectable procedures.

The MRP-VM concept within ACHILLES is an attempt to generalize this control layer. MRP stands for a meta-rational processing idea: the runtime should not assume one reasoning regime. It should retrieve only the knowledge relevant to the current state, condense that state into a smaller control nucleus, construct bounded frames for candidate regimes, route the task, execute locally, and reintegrate results into a common representation. A regime can be prompt-oriented, code-based, symbolic, retrieval-driven, human, or mixed. The runtime records why a regime was selected and which evidence it returned.

This architecture is more ambitious than ordinary orchestration. A workflow engine chooses nodes according to a graph. An agent chooses tools according to context. MRP-VM is intended to choose not only the next operation but the appropriate mode of interpretation and judgment. It asks whether the task should be handled as formal verification, causal analysis, literature synthesis, heuristic search, simulation, or human deliberation. It can also decide that the current representation is inadequate and request a change at a higher specification level.

The concept should be treated cautiously. A generalized meta-rational runtime risks becoming a vocabulary that merely renames routing. Its value must be demonstrated against simpler baselines: a monolithic prompt, a fixed workflow, a ReAct loop, and a conventional agent graph. The tests should measure not only task accuracy but semantic fidelity, evidence quality, cost, latency, selective recomputation, critic independence, and failure localization. MRP-VM becomes a contribution only if its explicit regime model produces practical gains.

SOP Lang Plan, and Pipeline are complementary layers. Plan represents compact stepwise coordination where the next task may depend on interaction. Pipeline represents heavier dependency graphs and build-like workflows. MRP-VM supplies execution semantics for nodes whose realization differs. The same IR can support planning, pipeline execution, analysis, and audit. This unification is one of the central ACHILLES hypotheses.

The industrial boundary should remain clear. Structured outputs, agent frameworks, MCP, skills, coding agents, spec-driven environments, and workflow systems are existing mechanisms with

varying maturity. The ACHILLES contribution is not the invention of agents, skills, or pipelines. It is the proposal to combine them around a specification-first intermediate representation and to generalize skills into evidence-bearing execution regimes governed by a meta-rational runtime. The research claim is that this combination can reduce reliance on opaque global LLM judgment while preserving adaptive capability.

The likely adoption path is incremental. A laboratory will not begin by deploying a universal VM. It may start with a versioned research specification and a small set of scientific skills: literature-grounding, protocol conformance, statistical checks, citation verification, and reproducibility packaging. These are composed in explicit pipelines. As contracts stabilize, the runtime can select among implementations based on cost and evidence. The same validation responsibility might be performed by an LLM subagent in an exploratory phase, deterministic code for routine cases, and a human specialist for consequential ambiguity. The workbench learns not only how to perform the task but which regime is appropriate.

This is the practical meaning of interpreters as intermediate steps in specification-driven development. They convert an abstract commitment into a local operation without surrendering global control. They allow symbolic processing where symbolic processing is faster and stronger, and use models where language and open-ended synthesis are indispensable. They make heterogeneity a designed property rather than an accidental collection of tools.

Table 4. Existing industrial mechanisms and the ACHILLES research hypothesis

What industry increasingly provides	What the ACHILLES direction adds as a research hypothesis
Structured generation through schemas, grammars, types, constrained decoding, and language-model programming	A common intermediate representation in which specifications, constraints, provenance, routes, candidate assemblies, and results coexist.
Agent frameworks that expose tools, memory, routing, subagents, and graph-based orchestration	SOP Lang Plan and Pipeline as explicit long-lived artefact graphs rather than only transient control flow.
Coding-agent skills and reusable instruction packages	Generalized skills as bounded execution regimes with applicability, evidence obligations, validators, costs, and reintegrable outputs.
Context protocols and tool registries	Regime-sensitive interpretation through an emerging MRP-VM that can route between LLMs, deterministic code, retrieval, solvers, and domain validators.
Specification-oriented development environments	The Specification Ladder and Mirroring Pattern as a control mechanism for semantic drift across successive levels of abstraction.

The right column describes a proposed research programme, not a claim of universal production maturity.

4.3 Independent critics, evidential escalation, and decision rights

A workbench that generates and executes without a strong criticism architecture will automate confidence rather than knowledge. The generator and certifier must therefore be separated structurally. Separation by prompt is not enough. Telling the same model to “be critical” after it has constructed a solution can improve local quality, but it does not create independence. The system needs distinct roles, evidence types, dependencies, and authorities.

The generator proposes claims, representations, plans, code, and experiments. The executor runs tools and records outputs in controlled environments. Critics evaluate bounded properties. An adjudicator changes claim status. These functions can share infrastructure, but their artefacts and responsibilities should remain distinct. The generator should not be able to promote its own claim by producing more persuasive prose.

Communication should occur through typed evidence packets rather than unconstrained debate alone. A packet identifies the claim, scope, assumptions, specification links, artefact identities, environment, executed operations, results, known failures, and the question assigned to the critic. The critic returns the method used, verdict, confidence boundary, counterexample or limitation, dependencies, and proposed next decisive test. A free-form explanation can accompany the packet, but the structure makes omissions visible.

Initial critic reports should be committed before cross-critic discussion. This preserves disagreement and reduces premature convergence. After reports are fixed, critics may inspect one another's evidence, revise their positions, or request additional experiments. The workbench records both the initial and revised judgments. Consensus is then traceable rather than manufactured through averaging.

Critic diversity should be functional. A formal critic checks logical validity. An implementation critic checks whether code realizes the claimed method. A statistical critic examines uncertainty and assumptions. A causal critic asks whether the design supports the interpretation. A robustness critic explores distribution shifts, adversarial cases, and scaling. A novelty critic searches prior work. A domain critic judges whether the operationalization matters. A governance critic checks risk, consent, and release conditions. No critic should be presented as competent outside its declared scope.

Evidence should advance through stages of increasing independence and consequence. A practical ladder begins with an idea or analogy. The next stage is a specified, literature-grounded claim. The third is an executable artefact with recorded environment and tests. The fourth is independent reproduction. The fifth adds adversarial, scaling, and negative-control testing. The sixth adds accountable specialist review. The seventh is external or deployment-specific validation. These stages are not a universal scalar truth measure. They prevent different forms of evidence from collapsing into one confidence label.

The ladder also supports different release thresholds. An exploratory conjecture may be useful at an early stage if uncertainty is explicit. A software method may be shared once it is executable and reproduced, while claims of general superiority remain provisional. A clinical recommendation cannot rely on internal reproduction alone. A theorem may reach strong formal status while its relevance to an application remains weak. Claim type and intended consequence determine which evidence stages are necessary.

Causal attribution deserves a dedicated critic because sophisticated systems frequently earn credit for effects produced by simpler factors. The critic constructs matched baselines and removal tests. It asks whether the capability survives replacement of the proposed mechanism with the simplest carrier that preserves the evaluator's observations. It checks whether data, compute, preprocessing, or evaluation changed. It tries to separate "the system works" from "the explanation is correct." Automated research is especially well suited to this task because code substitution and reruns can be cheap, provided the specification requires them.

Decision rights connect the ladder to governance. A low-risk transition from idea to executable prototype can be policy-driven. Promotion to a public claim may require an author. Promotion to a clinical or safety-relevant recommendation may require institutional review and external evidence. The workbench must represent not only who can execute a step, but who can authorize a status change. Capability and authority are different.

The same principle applies to stopping. An agent may be capable of running another experiment, but the budget owner may decide that the expected information value is too low. A model may produce a dual-use procedure, but the safety authority may block release. A reviewer

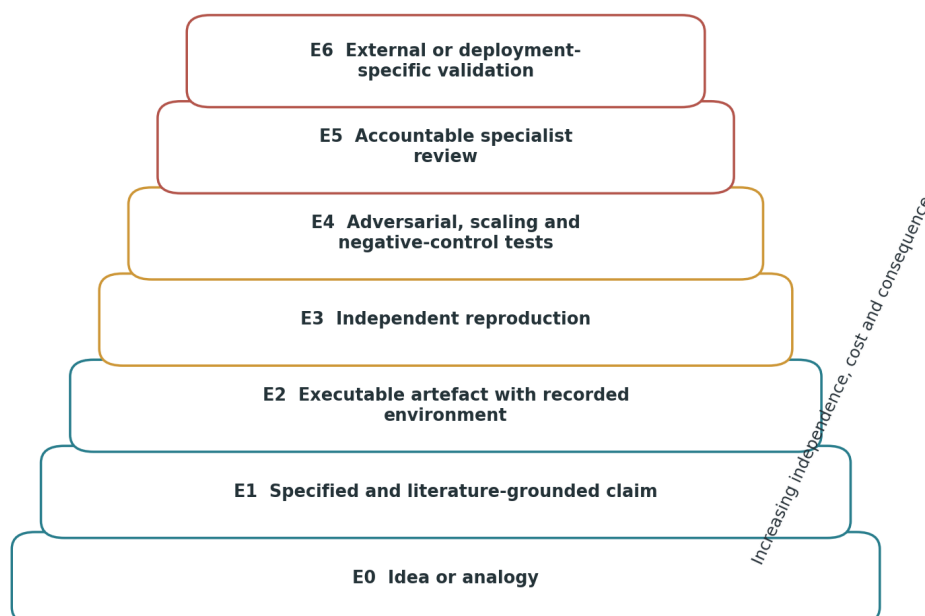
may request data that participants did not consent to share. The runtime should treat these decisions as legitimate terminal states, not obstacles to be circumvented.

Machine-assisted peer review fits naturally into this architecture. AI critics can perform preflight checks before submission, reducing avoidable burdens on reviewers. During review, they can test claims, inspect provenance, compare code and prose, or suggest counterexamples. Human reviewers decide which issues matter, assess significance and fairness, and take responsibility for recommendations. After publication, the same infrastructure can attach replication and criticism continuously. Review becomes a lifecycle rather than a one-time verdict.

The architecture also reduces the temptation to build one omniscient reviewer model. A single model is attractive because it is easy to deploy and produces coherent reports. Its coherence can hide shared failure. A portfolio of specialized critics with explicit dependencies is more cumbersome but epistemically stronger. Where critics disagree, the workbench should escalate the exact point of disagreement rather than ask a larger model to synthesize a smooth answer.

The final design principle of this chapter is that evidence and authority must be explicit at every promotion boundary. Local autonomy is valuable because it expands search and reduces procedural cost. Global evidence is necessary because local success can be misleading. Decision rights are necessary because scientific outputs enter social worlds. The workbench is credible only when these three layers remain connected without being collapsed.

Evidential escalation



Levels are claim-specific stages, not a universal scalar of truth.

Figure 6. Evidential escalation. Promotion requires new types of support, not repeated confidence from the same generator.

Table 5. Evidential escalation from candidate to released claim

Evidence level	Required change in support
E0 - Candidate	A hypothesis, mechanism, proof sketch, model, or interpretation exists, but no competent check has yet supported it.

Evidence level	Required change in support
E1 - Internally coherent	Basic type, syntax, consistency, and dependency checks pass; this eliminates malformed candidates, not false scientific claims.
E2 - Executed	The proposed computation, protocol, or experiment has run and produced recorded outputs under a declared environment.
E3 - Challenged	Negative controls, ablations, counterexample search, alternative explanations, or adversarial critics have tested the central inference.
E4 - Independently reproduced	A materially independent executor, environment, dataset, laboratory, or method obtains compatible evidence.
E5 - Specialist adjudicated	Qualified human or institutional reviewers evaluate significance, validity, risk, and unresolved disagreement.
E6 - Released under policy	The result meets the domain-specific threshold for publication, deployment, clinical use, regulation, or another consequential action.

Chapter 5. Methodological Rigor Across the Sciences

5.1 Formal and computational sciences: strong oracles, fragile formalizations

Automation advances fastest where a proposal can be tested by a strong, inexpensive, executable oracle. Formal mathematics, algorithm design, software engineering, and portions of machine learning offer the clearest examples. A proof checker can reject an invalid derivation. A compiler can reject malformed code. A test suite can expose regressions. A programmatic evaluator can score an algorithm over many instances. These domains are therefore likely to produce the earliest systems with substantial local autonomy.

The advantage should not be overstated. An oracle is strong only for the property it actually checks. A theorem prover can certify that a formal statement follows from axioms, but it cannot determine by itself whether the statement faithfully represents the informal conjecture or the intended application. A test suite can show that code passes selected cases, not that the requirements are complete. A benchmark can measure performance on a dataset while failing to capture robustness, cost, fairness, or operational relevance. The central methodological risk is specification error: the system becomes extremely reliable at solving the wrong formal problem.

Formal mathematics therefore requires a dual chain of rigor. The first chain is syntactic and deductive. Definitions, lemmas, theorem statements, proof terms, and trusted kernels must be versioned and checkable. The second is semantic and interpretive. The mapping from the human conjecture to formal objects must be reviewed, and the scope of the theorem must be communicated without expansion. A model can help translate informal arguments into Lean, Isabelle, Coq, or another proof assistant, but the translation itself is a claim. The research object should preserve the original statement, formalization decisions, unresolved mismatches, and the exact proof environment.

AI-assisted theorem proving is most credible when the generative model is a proposer under a small trusted kernel. The model can search for lemmas, tactics, constructions, and counterexamples. Failed proposals are cheap. The kernel determines formal validity. Human mathematicians remain essential in selecting important conjectures, judging explanatory value, designing representations, and verifying that formal statements capture the intended concepts. A proof can be correct and scientifically unilluminating. Executable science should make this distinction visible rather than allowing formal success to stand in for significance.

Program search follows a similar pattern. FunSearch and AlphaEvolve demonstrate that language models can generate program variants while evaluators and evolutionary selection provide external pressure (Romera-Paredes et al., 2024; Novikov et al., 2025). This architecture is powerful when the objective is executable and relatively complete. Its methodological obligations include evaluator validity, test-set separation, resource accounting, and analysis of whether the discovered program exploits an implementation detail. A candidate should be tested across distributions, scales, and independent implementations. Where possible, formal analysis should complement empirical score.

Software research adds a second layer: implementation fidelity. A paper may describe one method while the code implements another. Dependencies, random seeds, hardware, numerical precision, preprocessing, and data leakage can alter results. Executable rigor therefore requires environment capture, build reproducibility, tests tied to specification clauses, and machine-readable

experiment manifests. Continuous integration should rerun relevant analyses when code or data change. A claim-evidence graph should distinguish successful execution from agreement with the described method and from external reproduction.

The rise of coding agents offers a preview. Agents can navigate repositories, implement features, run tests, and revise code over long trajectories. Yet PaperBench shows that end-to-end replication remains difficult even in a fully digital field, and developer-productivity studies show that subjective speed does not always match measured completion time (Starace et al., 2025; METR, 2025, 2026). Scientific coding is harder than ordinary feature work because the specification may be incomplete and the code is itself part of the evidence. The workbench must expose agent actions as reviewable changes rather than treating a passing run as proof of scientific fidelity.

Machine learning requires especially careful rigor because benchmark feedback is rapid and optimization pressure is intense. A valid research object should identify data lineage, train-validation-test boundaries, search budgets, model and library versions, compute, hyperparameter policy, stopping rules, uncertainty across seeds, and all baselines. It should record failed runs and the extent of researcher or agent adaptivity. Claims about generalization require distributional tests; claims about mechanisms require interventions or causal analyses; claims about efficiency require matched hardware and quality; claims about safety require adversarial and worst-case evaluation.

Automated ML research systems can generate many variants and papers, which makes selective reporting a central risk. The workbench should treat every experiment generated under the active specification as part of the portfolio, even if only a subset appears in the article. This does not require publishing every transient run, but it requires accounting for the search process so that apparent significance is not interpreted as if one hypothesis had been tested once. Test-time compute used to generate and select research ideas should be reported as part of the method.

Formal methods can also strengthen agentic research infrastructure itself. State machines can constrain tool use. SMT solvers can check resource and dependency conditions. Type systems can enforce claim and evidence schemas. Temporal logic can express safety properties for laboratory controllers. Property-based testing can generate edge cases from specifications. These techniques should replace LLM judgment where their semantics are adequate. Their inclusion is not a retreat from intelligence; it is an allocation of intelligence to the layer where ambiguity actually remains.

The probable future in formal and computational sciences is high local autonomy inside strong verification envelopes. Agents will propose proofs, algorithms, code, and experiments. Pipelines will execute and score them. Independent systems will reproduce selected results. Humans will increasingly work at the levels of problem choice, formalization, evaluator design, explanatory synthesis, and release. The limit will not be the generation of candidate artefacts. It will be assurance that the formal and executable objects correspond to questions worth answering.

5.2 Physical and life sciences: instruments, calibration, and resistance from the world

Physical and life sciences introduce a form of external resistance that cannot be simulated away. Materials fail to synthesize. Biological systems vary. Instruments drift. Reagents degrade. Samples are contaminated. Environmental conditions matter. A model can propose a perfect protocol, but the world returns observations that do not respect the model's narrative. This resistance makes autonomous laboratories scientifically promising and methodologically demanding.

Self-driving laboratory research has already shown that automation can connect planning, robotic execution, measurement, and active learning. Systems in chemistry and materials science

select experiments, control instruments, update models, and search large design spaces more efficiently than manual iteration in bounded tasks (Häse et al., 2019; Burger et al., 2020; Szymanski et al., 2023). Recent multi-agent platforms add natural-language planning and recovery, while automated instrument systems demonstrate long-duration operation in specialized settings (Panapitiya et al., 2026). These results support partial autonomy, not a universal laboratory scientist.

Rigor begins with the distinction between protocol and actual execution. A protocol states intended operations. An execution record captures what the instrument did, when it did it, under which calibration, with which materials, and with what deviations. The research object should preserve both. A laboratory step cannot be considered reproduced because the same text was followed if instrument state, reagent lot, environmental conditions, or measurement pipeline changed materially.

Calibration is therefore an epistemic dependency. Instruments do not produce naked facts. They produce signals under models of measurement. Calibration standards, maintenance events, uncertainty estimates, and preprocessing transformations should be linked to the claims that depend on them. If a later calibration correction changes a measurement, the workbench should identify affected analyses and claims. This is the physical-science analogue of a software dependency update.

Negative controls and positive controls must be first-class nodes. An automated system optimized for throughput may treat controls as overhead because they do not directly improve the target score. Specification-driven inquiry treats them as evidence-producing branches with their own acceptance criteria. A claim cannot advance because the target measurement changed if the control structure cannot distinguish mechanism from contamination, drift, or generic response.

Active learning and Bayesian optimization are valuable because experiments are costly. They can select measurements expected to improve a model or objective efficiently. Yet the acquisition function is a proxy. It can prefer regions with predictable gains, avoid anomalous regimes, or exploit model misspecification. The workbench should monitor diversity, uncertainty calibration, constraint violations, and the possibility that the active learner is steering away from observations that would falsify its representation. A portion of the budget should often be reserved for adversarial or coverage-oriented experiments rather than pure optimization.

Biology adds heterogeneity, hierarchy, and path dependence. Results can depend on cell line, organism, batch, developmental stage, microbiome, environment, and laboratory practice. Automated hypothesis systems can search literature and public data, propose mechanisms, and design analyses. Robin and Co-Scientist illustrate how such systems can contribute to hypotheses that receive experimental attention (Ghareeb et al., 2026; Gottweis et al., 2026). The evidential status of these examples is strongest where independent laboratory work tests a specific proposal and weakest where model ranking or plausibility is treated as validation.

A biologically rigorous workbench should preserve the chain from claim to specimen and measurement. Sample provenance, inclusion criteria, randomization, blinding, batch structure, assay versions, quality-control decisions, and deviations matter. Statistical models should be linked to the biological level at which inference is claimed. An effect in a cell line does not automatically support an organism-level mechanism; an organoid result does not establish clinical benefit. The evidence ladder must make translational distance explicit.

Mechanistic claims require interventions or converging evidence. Automated systems are likely to excel at generating plausible pathways because the literature is rich and language models can connect distant findings. This strength increases the need for causal discrimination. Knockout, perturbation, dose-response, temporal sequencing, orthogonal measurement, rescue experiments,

and matched alternatives should be represented as obligations appropriate to the mechanism. A workbench can help design and track these tests, but domain experts must judge whether they are biologically decisive.

Safety changes the autonomy envelope. A computational branch can often be rerun after failure. A physical experiment may consume scarce material, damage equipment, expose personnel, affect animals, or create environmental or dual-use risk. The runtime therefore needs hard permissions, hazard classifications, instrument interlocks, transaction logs, and emergency stop mechanisms. A language model should not be the final authority on whether an action is safe. It can classify and explain, but deterministic rules and responsible humans must govern high-consequence transitions.

Robotic science also requires recovery semantics. Physical workflows are not reliably idempotent. Repeating a failed step may produce a different state or compound damage. The plan must represent irreversible actions, checkpoints, compensating procedures, and the uncertainty of current state. When a robot loses track of a sample or an instrument reports an anomaly, the correct action may be to pause and request inspection rather than improvise.

Reproducibility in experimental science is therefore not a single “rerun” button. It is the ability to reconstruct enough of the material, procedural, measurement, and analytical context for another site to test the claim. The workbench can reduce friction by packaging protocols, instrument metadata, software, and evidence. Inter-laboratory reproduction remains a distinct evidential stage because local automation can make one laboratory internally consistent while preserving a site-specific bias.

The probable future is a network of partially autonomous laboratories rather than one general robotic scientist. Each facility will expose machine-readable capabilities, constraints, calibration states, and evidence profiles. Research workbenches will allocate experiments across sites, compare results, and trigger replication when common-mode risk is high. Human laboratory scientists will shift toward designing assays, maintaining trustworthy instrumentation, interpreting anomalies, and governing risk. The world will remain the final source of resistance, but the path from its signals to public claims will become more computationally explicit.

5.3 Clinical, social, and interpretive sciences: causality, legitimacy, and plural evidence

The most difficult domains for research automation are not necessarily those with the least data. They are those in which observations are entangled with values, institutions, strategic behaviour, historical context, and consequences for people. Clinical medicine, public health, economics, social science, and interpretive disciplines require rigor that cannot be reduced to execution success.

Clinical research combines empirical uncertainty with ethical obligation. A system can search trials, generate hypotheses, simulate designs, draft protocols, monitor data quality, and perform statistical analyses. It can identify drug candidates or patient subgroups. Yet the legitimacy of clinical research depends on consent, equipoise, privacy, professional responsibility, and a chain of regulation. A predicted benefit does not authorize intervention. The workbench must encode who may access data, who may change a protocol, how adverse events are handled, and which evidence threshold applies to each stage from exploratory analysis to clinical recommendation.

The target of inference must be explicit. Clinical studies often fail not because computation is weak but because the estimand is unclear, missing data are informative, treatment adherence varies, outcomes are selected after observation, or the study population differs from the population of use. Specification-driven inquiry should require a causal question, population, treatment strategy, outcome, time horizon, and handling of intercurrent events. Statistical analysis is then evaluated against the estimand rather than chosen for convenience.

Causal inference supplies concepts that an executable workbench can operationalize: directed acyclic graphs, identification assumptions, exchangeability, positivity, consistency, sensitivity analysis, and target-trial emulation (Pearl, 2009; Hernán and Robins, 2020). These tools do not automate truth. They make assumptions explicit enough to criticize. A causal interpreter can propose identification strategies and diagnostics, but a domain expert must judge whether the variables and interventions correspond to reality.

External validity and distributional consequences are equally important. A model may perform well on average while harming a subgroup, or a treatment effect may not transport across health systems. The evidence graph should preserve population boundaries, subgroup uncertainty, and data provenance. Release decisions should distinguish scientific evidence from clinical authority. A research finding can be publishable while remaining inappropriate for deployment.

Social science adds reflexivity. People change behaviour in response to measurement, policy, and prediction. Categories can be contested or politically consequential. Administrative data reflect institutions rather than neutral reality. Automated systems can process large corpora, generate survey instruments, estimate models, and compare theories, but they can also amplify the assumption that what is legible is what matters. Rigor requires conceptual validity, measurement invariance, design-based or causal identification, transparent coding, robustness to alternative operationalizations, and attention to whose experience is absent.

The research specification should include normative commitments that ordinary pipelines omit. What outcomes are valued? Which distributional effects matter? Who bears risk? Which groups have standing to contest categories or uses? These are not parameters that a model can infer from data without political choice. The workbench can make them explicit, versioned, and reviewable. It cannot make them value-free.

Economics and policy research illustrate the interaction between model execution and institutional consequence. A model may be internally coherent and empirically calibrated yet depend on behavioural assumptions or equilibrium effects that fail under intervention. Automated simulation can explore scenarios and sensitivity, but policy claims require identification, robustness, distributional analysis, and accountability. A system should distinguish forecasting accuracy from causal policy evaluation and both from the legitimacy of adopting a policy.

Qualitative and interpretive research presents a different challenge. The absence of a deterministic oracle does not imply the absence of rigor. Claims can be grounded in sources, quotations, archival context, coding procedures, triangulation, counter-interpretations, and reflexive analysis. A workbench can link interpretations to evidence, preserve coding revisions, compare readings, and expose the argumentative path. It should not pretend that a consensus score resolves meaning.

Language models are powerful in these disciplines because they can summarize, translate, classify, and propose interpretations. Their fluency is also dangerous. They can normalize a dominant reading, invent context, or erase ambiguity. An interpretive research object should preserve primary evidence and dissenting views. Machine-generated classifications should be treated as coding proposals with uncertainty and provenance. Human scholars remain responsible for contextual judgment, theoretical significance, and the ethical treatment of persons and communities represented in the material.

Historical research shows why versioned evidence matters. New archives can change an interpretation without making the previous argument fraudulent. Competing narratives may remain live. A research object can represent which sources support each claim, which absences are material, and how an interpretation changed. It should allow plural views rather than enforcing one canonical account.

Peer review in these domains must therefore evaluate argumentative integrity and contextual adequacy, not only execution. Automated preflight can detect citation errors, missing source links, inconsistent coding, or statistical mistakes. It can generate counterarguments and search for neglected literature. The final judgment about significance, fairness, and interpretation belongs to a community of accountable readers. The workbench should make their disagreements durable.

Across clinical, social, and interpretive fields, the central rule is that computational verifiability must not be confused with epistemic closure. The more a claim affects people, the more the system must represent legitimacy, consent, distribution, and contestability. Executable science can strengthen these domains by making evidence and decisions traceable. It becomes harmful when it converts contested concepts into hidden technical parameters and then presents the output as neutral.

The order of automation across sciences will therefore depend on two variables: the strength of the evaluator and the latency of validation. Formal proof has a strong, fast oracle once formalization is complete. Software and computational experiments have fast but incomplete tests. Laboratory science has stronger external resistance but slower, costlier feedback. Clinical and social interventions have long latency, ethical constraints, and contested objectives. Interpretive disciplines have plural standards of evidence and no universal execution metric. Autonomy should be inversely proportional to unresolved ambiguity and consequence, not proportional to model confidence.

A scientifically mature workbench will not impose one methodology. It will carry domain-specific rigor profiles. These profiles define claim types, required evidence, applicable interpreters, admissible automation, and release authority. The common substrate is specification, provenance, evidence, and versioning. The standards of judgment remain irreducibly plural.

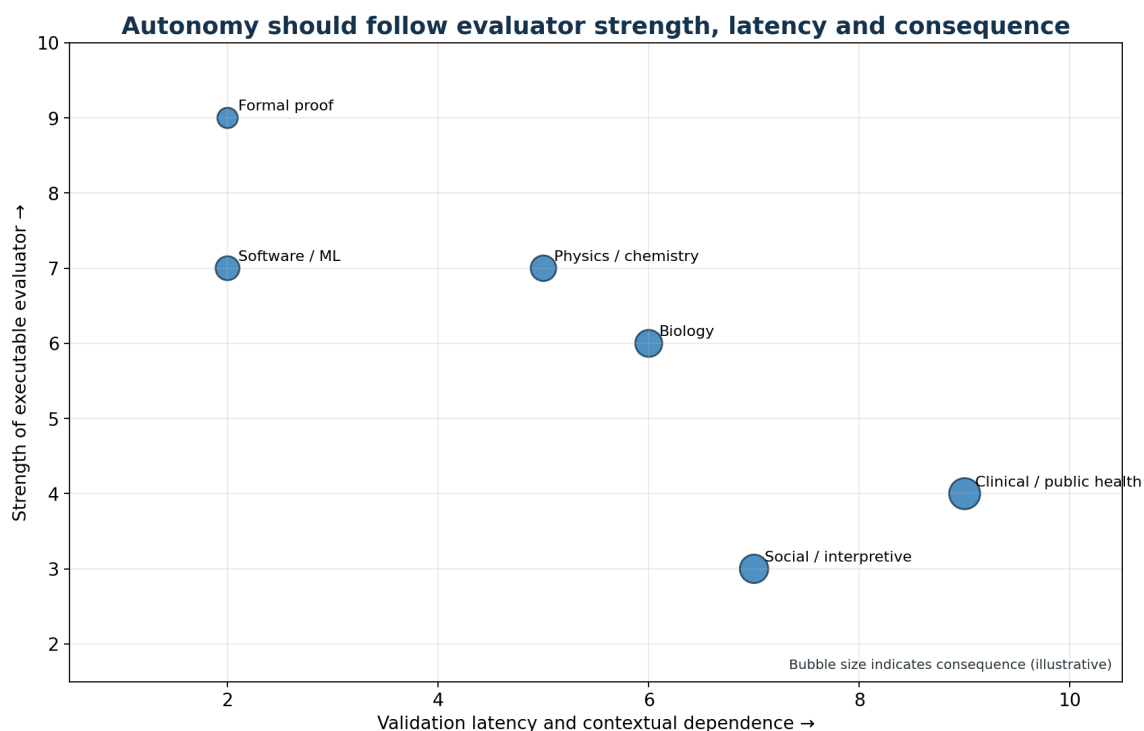


Figure 7. Domain-sensitive autonomy. The coordinates and bubble sizes are illustrative; the governing variables are evaluator strength, validation latency, and consequence.

Table 6. Domain-sensitive rigor and admissible autonomy

Domain family	Methodological obligations and realistic autonomy
Formal mathematics and verified systems	Proof kernels and executable specifications permit strong local automation, but the formal statement must faithfully represent the intended theorem or requirement.
Software, machine learning, and computational science	Tests, benchmarks, static analysis, and reruns support rapid loops; dataset leakage, benchmark gaming, environment drift, and claim inflation require independent evaluators.
Physics and engineering	Simulation can automate exploration, but calibration, uncertainty propagation, dimensional checks, safety margins, and contact with measured reality remain decisive.
Chemistry and materials science	Robotics and active learning support closed loops where instruments are standardized; synthesis failures, impurities, characterization quality, and lab safety constrain autonomy.
Biology and preclinical research	Automated protocols can scale assays, yet biological variability, batch effects, contamination, construct validity, and multilevel causation demand layered controls.
Clinical medicine and public health	Decision support is plausible; autonomous evidential promotion is limited by confounding, heterogeneity, consent, distributional effects, regulation, and the cost of error.
Social science and economics	Computational exploration can expand models and replications, while causal identification, measurement validity, strategic behaviour, institutions, and normative stakes require plural adjudication.
Interpretive and historical disciplines	Automation can search, compare, translate, and trace evidence; rigor is achieved through accountable argument, source criticism, contextual competence, and contestability rather than algorithmic closure.

Chapter 6. The Human and Institutional Transition

6.1 From vibe coding to vibe research: the accumulation of epistemic debt

Coding agents provide the most visible preview of what happens when generative capacity enters a technical profession. A developer can describe a feature, accept a sequence of edits, run tests, and obtain a working system without understanding every line. This style is sometimes called vibe coding. The term is playful, but the underlying transition is serious: production can outrun comprehension.

The experience is neither simply good nor simply bad. Coding agents can reduce boilerplate, navigate unfamiliar repositories, propose tests, repair defects, and make programming accessible to people with less formal training. They can also generate brittle dependencies, security vulnerabilities, duplicated abstractions, and systems whose maintainers cannot explain why they work. Passing tests create confidence only to the extent that the tests embody the intended behaviour.

The analogous phenomenon in science is vibe research. A researcher receives a polished literature synthesis, plausible hypotheses, executable notebooks, attractive figures, statistical interpretations, and manuscript prose. The workflow feels fast because visible artefacts accumulate. Yet the difficult work may remain unresolved: whether the literature search missed negative evidence, whether the operational definition matches the concept, whether the baseline is fair, whether the code implements the claimed method, whether the effect is causal, whether multiple testing has been accounted for, and whether anyone understands the failure conditions.

This creates epistemic debt. Technical debt is the future cost created by expedient software decisions. Epistemic debt is the future cost created when a research artefact is accepted before its assumptions, provenance, and validity are understood. The debt may be hidden because the result looks complete. It becomes visible when another team tries to reproduce it, when a new dataset produces a contradiction, when the system is deployed, or when a reviewer asks a question that no one in the team can answer.

AI can increase epistemic debt through semantic compression. A model translates a complex chain into a coherent explanation, smoothing uncertainty and disagreement. Each summary is useful, but repeated summaries can remove the conditions under which a claim is valid. The team begins to reason from the compressed narrative rather than the underlying evidence. This is why the research object must preserve links and why agents should receive bounded source artefacts rather than only previous summaries.

The productivity evidence from coding is instructive. METR's randomized study of experienced open-source developers using early-2025 AI tools found that participants took longer on assigned tasks despite believing they had been faster; later observations suggested improvement in tool capability but also serious selection and measurement complications (METR, 2025, 2026). The result should not be universalized. Coding tools change rapidly, tasks vary, and experienced maintainers may differ from other users. Its importance lies in the divergence between perceived and measured productivity.

Science is especially vulnerable to this divergence because completion is hard to define. A manuscript draft can create the perception of progress even when evidential work has not begun. A generated hypothesis can feel like discovery before novelty is established. A running analysis can

conceal an invalid estimand. The more fluent the output, the greater the risk that appearance substitutes for task completion.

A workbench should therefore display debt. It should show unresolved assumptions, unverified sources, unreplicated claims, missing controls, environment drift, common dependencies among critics, and branches whose stopping conditions are undefined. A progress indicator should not report only how many tasks are complete. It should report the evidential state of the claims those tasks serve.

This changes the interface. Chat interfaces encourage a conversational sense of movement. Every response appears to advance the project. An executable-science interface should sometimes resist. It should show that the last three model outputs did not add a new evidence type, that a claim has been rewritten more confidently without changing support, or that a proposed experiment cannot distinguish the live hypotheses. It should make epistemic stagnation visible even when textual production is high.

Human comprehension cannot scale linearly with generated volume. The response is not to require that every researcher understand every artefact in full. Modern science is already team-based and distributed. The response is scoped ownership. Every consequential claim must have someone who understands its specification, evidence, and limitations well enough to accept responsibility. Specialists can certify bounded components. The workbench records these scopes and dependencies.

The educational consequence is substantial. Training that focuses only on producing code, analyses, or prose will be weakened by automation. Future researchers need stronger competence in problem formulation, evaluator design, causal reasoning, validation, uncertainty, provenance, and the diagnosis of tool failure. They need to read generated work adversarially and to distinguish formal correctness from scientific relevance. The ability to ask what would change one's mind becomes more important than the ability to draft a conventional methods section from memory.

Vibe research is not avoided by banning AI. It is avoided by changing the unit of work from plausible output to evidence-bearing change. A generated artefact becomes useful when it is connected to a specification, validated under a declared regime, and accepted by someone with the relevant competence. The goal is not to slow every operation. It is to prevent speed at the surface from hiding debt in the structure.

6.2 Deskillling, credit, peer review, and scientific monoculture

Every automation changes the distribution of competence. Calculators reduced the need for manual arithmetic while increasing the range of feasible calculation. Statistical software democratized complex methods while making it possible to run analyses without understanding their assumptions. Search engines changed memory and discovery. Coding agents and scientific copilots will continue this pattern, but the breadth of generative systems makes the transition unusually general.

Deskillling is not simply loss of knowledge. It is the relocation of knowledge from people into infrastructure. This can be beneficial when infrastructure is reliable, transparent, and widely accessible. Few researchers need to implement a linear algebra library from first principles. The danger arises when essential competence moves into opaque, concentrated systems that cannot be inspected, repaired, or contested locally. A laboratory may become faster while losing the capacity to recognize when its agent, model, data source, or instrument interface fails.

The appropriate response is not to preserve every manual practice. It is to identify irreducible supervisory competence. A researcher using an automated causal pipeline must understand the

estimand and identification assumptions. A biologist using a protocol agent must recognize control failures and biological implausibility. A mathematician using proof automation must inspect formalization boundaries. A reviewer using an AI critic must know when the critic's dependency or rubric is inadequate. Training should preserve the ability to diagnose and take over, not the ritual performance of tasks that machines perform more accurately.

Institutional incentives can accelerate the wrong form of deskilling. If universities reward paper count while AI lowers the cost of papers, researchers will be pushed toward production and away from the slow acquisition of judgment. Junior scientists may become operators of systems whose assumptions are set by vendors or senior laboratories. The visible achievement will be output; the invisible loss will be the capacity to formulate independent questions and design decisive tests.

Credit systems must change accordingly. Authorship currently compresses many kinds of contribution into a name order. In automated research, that compression becomes less defensible. The person who writes the first draft may contribute less than the person who defines the specification, builds a validator, performs an independent replication, closes a costly branch, or maintains the evidence object. Machine generation will make prose and routine code less scarce. Credit should migrate toward epistemic labour.

A versioned research object makes this migration technically possible. It can attribute specification decisions, artefacts, reviews, replications, and maintenance. Contribution statements can be generated from committed actions and then reviewed by the team. This should not become a surveillance-based productivity score. Attribution is useful when it supports recognition and responsibility; it becomes harmful when every micro-action is converted into ranking.

Peer review is another institution designed for a slower production regime. Submission volume already strains reviewer attention. Automated paper generation could overwhelm the system with work that is individually plausible but collectively low value. An automated reviewer alone cannot solve the problem because it may share the generator's biases and because editorial judgment includes significance, fairness, conflict, and responsibility.

The more credible transition is continuous machine-assisted criticism. Before submission, a workbench checks reproducibility, citations, claim-code consistency, statistical reporting, missing baselines, and provenance. Authors see and respond to these reports. During formal review, reviewers receive targeted evidence packets and can invoke additional checks. After publication, replication and criticism attach to the same research object. AI reduces avoidable review labour while human reviewers focus on the issues that require judgment.

The randomized ICLR review-feedback study provides an early example. The system did not replace reviewers or decide acceptance. It identified vague, mistaken, or unprofessional comments and offered suggestions; a meaningful fraction of reviewers revised their reports, and the revised reports were judged more informative (Thakkar et al., 2026). This is a desirable division of labour: AI improves the quality of a human institutional act without pretending to own the act.

Automated review also creates risks. Authors may optimize papers to a known reviewer model. Journals may use opaque screening systems to reject unconventional work. Reviewers may defer to machine-generated criticism they have not checked. Proprietary models may encode hidden disciplinary or linguistic biases. A defensible system therefore records the critic version, evidence, scope, and conflicts. Authors must be able to contest machine findings. Editorial authorities remain accountable.

Scientific monoculture is a subtler risk. AI systems learn from existing literatures and benchmarks. They are especially strong where data and accepted evaluators are abundant. This creates an economic pull toward mature, well-instrumented fields and familiar forms of evidence. Hao et al.'s large-scale study reports that AI-augmented scientists achieved higher individual output

and impact while the collective focus of science narrowed (Hao et al., 2026). Whether every measured association is causal, the mechanism is plausible: automation amplifies areas that already have machine-readable structure.

Specification explosion can counter this tendency only if diversity is deliberate. A portfolio should include alternative representations and neglected hypotheses, but model-generated diversity is not necessarily intellectual diversity. Different agents may produce variations around the same training distribution. Institutions must preserve funding and evaluation paths for questions with weak current benchmarks, scarce data, local knowledge, or long validation cycles. The workbench should measure dependency diversity and unresolved dissent, not merely the number of branches.

This issue has geopolitical and linguistic dimensions. If research copilots are trained primarily on English-language, high-impact, well-digitized literatures, they can further marginalize local journals, tacit knowledge, and communities with limited data infrastructure. A scientific workbench should expose corpus scope, support multilingual retrieval, and allow domain communities to define evidence profiles. Open standards and local competence are necessary to prevent a few model providers from becoming the invisible methodology of global science.

The human transition is therefore not a simple movement from researcher to supervisor. It is a reallocation of work among specification, generation, validation, interpretation, maintenance, and governance. Some researchers will build scientific interpreters and evidence infrastructure. Others will specialize in domain criticism, experiment design, or cross-disciplinary translation. Teams may become smaller for some computational projects and larger for high-stakes validation. The status system should reward those who make automated research reliable, not only those who release its visible outputs.

6.3 Governance, privacy, and the legitimate role of slowing down

Automation is often described as a struggle against friction. In research, some friction is waste: manual format conversion, duplicated analysis, inaccessible data, repeated literature searches, and bureaucratic reporting. Other friction is epistemic or ethical: independent review, consent, calibration, replication, cooling-off periods, and public deliberation. A mature automation programme must distinguish them.

The coding-agent analogy is useful again. Continuous deployment can move software changes into production rapidly, but high-reliability organizations add tests, staged release, permissions, observability, and rollback. The purpose is not to reject speed. It is to make speed conditional on evidence. Executable science requires an analogous release discipline.

Governance should be proportional to consequence. A low-risk computational conjecture can be shared with an executable artefact and explicit uncertainty. A model that influences clinical care, environmental intervention, public policy, or critical infrastructure requires stronger validation, named authority, and procedures for contest and correction. The same research object may support different release views: exploratory, peer-reviewed, regulatory, and operational. Evidence sufficient for one view may be insufficient for another.

Decision rights should be encoded explicitly. The person who can run an experiment is not necessarily the person who can release a conclusion. A model with tool access does not have authority because it has capability. A principal investigator may authorize a laboratory branch but not a clinical recommendation. A data steward may approve use under one purpose but not another. A safety officer may stop an action even when the scientific team prefers to continue. The runtime should enforce these distinctions.

Privacy extends beyond personal data. Research contains confidential ideas, reviewer identities, industrial information, security-sensitive methods, and records of provisional error. A workbench that logs everything for transparency can become coercive infrastructure. It can discourage speculation, expose vulnerable collaborators, and create a permanent record of unformed thoughts. Layered visibility and retention are therefore part of scientific freedom.

The public release layer should preserve what is needed to justify a claim: the specification version, supporting evidence, relevant provenance, known limitations, conflicts, and decision authority. Internal layers may contain richer detail under access control. Some evidence can be verified through secure computation, controlled environments, or independent attestations without releasing raw data. Privacy-preserving verification will be particularly important in medicine, social research, and proprietary industrial science.

Governance must also address model and infrastructure concentration. If one company supplies the model, retrieval layer, agent framework, cloud, and review system, dependency diversity becomes illusory. A failure or policy change can affect many institutions simultaneously. Open interfaces, portable research objects, local execution where feasible, and the ability to substitute models and validators are scientific resilience requirements.

Model Context Protocol and related standards help at the tool boundary, but interoperability must extend to evidence and identity. A research object should remain meaningful when moved between institutions. Claims, artefacts, provenance, and verdicts need stable representations. Domain policies can differ, but the object should declare which profile and authority apply. The future infrastructure is more likely to be federated than centralized.

Legitimate slowing occurs when additional time creates a new kind of evidence or allows accountable deliberation. Independent reproduction, long-term follow-up, safety testing, community consultation, and adversarial review cannot always be compressed by more compute. Their latency is part of the phenomenon. A clinical outcome may require years. An ecological intervention may have delayed effects. A social policy may change behaviour after adoption. The system should not substitute simulation confidence for elapsed evidence.

Slowing can also protect against synchronized error. Generative systems can propagate one interpretation through literature summaries, grant drafts, reviews, and policy documents at machine speed. Waiting for independent data or alternative analysis can be rational even when the initial evidence appears strong. A workbench can represent a claim as promising but held, with a stated release condition. The absence of immediate publication becomes an explicit governance choice rather than a failure to complete.

Not every delay is legitimate. Institutions can use review and compliance to protect incumbents, suppress criticism, or avoid responsibility. The workbench should make delays attributable and time-bounded. A hold should state who imposed it, why, what evidence can resolve it, and when it must be reviewed. Governance itself needs versioning and audit.

The ideal human role is therefore neither constant micromanagement nor ceremonial approval. Humans should intervene where the cost of semantic drift, weak validation, or illegitimate action is high. They should define purposes, contest representations, design discriminating tests, allocate scarce attention, interpret hard feedback, and accept responsibility. Automation should eliminate administrative burden and repeated mechanical work so that these functions receive more, not less, time.

The institutional success of executable science will be measured by whether it increases trustworthy correction. Can a claim be challenged without reconstructing the entire project? Can a laboratory stop an unsafe branch? Can a reviewer identify which evidence changed? Can a community preserve disagreement? Can a researcher explore privately and release responsibly?

Can an institution migrate away from a vendor without losing its scientific memory? These questions are as important as model benchmarks.

The lesson from coding agents is not that scientists should become product managers for machines. It is that generative speed changes the location of risk. When artefacts are cheap, the valuable human work moves toward specification, evaluation, integration, and responsibility. Institutions that continue to reward output volume will accumulate epistemic debt. Institutions that build evidence-bearing workflows may use automation to deepen rigor rather than to automate its appearance.

Chapter 7. What Is Possible, What Is Probable, and What Is Desirable

7.1 What is possible now: the scientific copilot as an evidence engineer

The capabilities required for a useful scientific copilot already exist in fragments. The gap is not the absence of a magical model. It is the lack of integration, domain contracts, reliable evaluation, and institutional adoption. A credible near-term system can be built today if its scope is narrower than the rhetoric of an autonomous scientist.

The strongest current use is literature and evidence engineering. A workbench can search across databases, construct a topic map, extract claims and methods, identify citation paths, compare terminology, and generate a structured evidence table. It can link each synthesized statement to supporting passages and flag conflicts or missing access. It can monitor newly published work and identify which claims in a project may need reconsideration. Human researchers remain responsible for source selection policy, interpretation, and significance, but the mechanical cost of maintaining a live literature model can fall substantially.

A second use is specification construction. Given a research objective, a copilot can help elicit claim types, assumptions, variables, constraints, baselines, risks, evidence thresholds, budgets, and stop conditions. It can detect ambiguity and produce a controlled-language projection. It can expand the specification into alternative hypotheses and tests. This is valuable even if the system does not generate a novel discovery. It makes the research programme explicit enough to criticize before expensive work begins.

A third use is computational execution. Coding agents can construct environments, implement methods, run tests, generate experiment manifests, compare results, and package artefacts. The workbench can require that every run identify the specification clause it serves and can capture code, data, model, and dependency versions. Independent agents or deterministic pipelines can reproduce selected results. Current reliability is insufficient for unsupervised release, but sufficient for accelerating bounded computational work under review.

A fourth use is validation preflight. The system can check citations, statistical reporting, code-prose consistency, missing baselines, data leakage indicators, environment completeness, and figure provenance. It can generate negative controls and robustness tests from claim types. It can produce a reviewer packet showing unresolved obligations. These functions are likely to deliver more immediate scientific value than automated paper writing because they reduce the cost of correction before publication.

A fifth use is formal and symbolic checking. Where a claim can be represented in a theorem prover, SMT solver, type system, or domain validator, the workbench can route it to a strong evaluator. Language models can translate and propose; symbolic systems can certify. Similar patterns apply to chemical constraints, schema validation, causal identification checks, and laboratory safety rules. The main requirement is to keep the boundary between formal validity and domain relevance explicit.

A sixth use is experiment planning and laboratory orchestration in bounded facilities. Existing self-driving laboratories show that active learning, robotics, and measurement can close loops for selected problems. A scientific workbench can add specification, evidence, provenance, and human gates. It can plan experiments, allocate instrument time, monitor execution, and update the portfolio. Full autonomy is neither necessary nor desirable for most laboratories, but partial automation can increase throughput and reproducibility.

A seventh use is peer-review assistance. Automated systems can perform pre-submission checks, improve reviewer comments, search for missing literature, and execute requested analyses. Reviewers can inspect machine-generated evidence and attach scoped verdicts. Journals can require a reproducibility and provenance profile without requiring one proprietary platform. The final recommendation remains human and institutional.

These capabilities support a concrete minimum viable scientific workbench. It needs a versioned research specification; stable identifiers for claims, artefacts, and evidence; a dependency graph; reproducible execution environments; a registry of interpreters or skills; a promotion policy; and release views. It can use ordinary Git, Markdown, containers, notebooks, workflow engines, provenance standards, and existing agent frameworks. The innovation lies in how these components are connected, not in replacing them all.

The most important near-term limitation is evaluator weakness. A copilot can generate more tests than the specification justifies, or produce a persuasive synthesis from an incomplete corpus. It can appear to validate its own assumptions. Therefore current systems should be positioned as evidence engineers and bounded collaborators, not autonomous authorities. Their outputs should be judged by trace completeness, error detection, reproduction, and specialist time saved, not by manuscript eloquence.

The second limitation is domain integration. Laboratories and research groups use heterogeneous data formats, instruments, policies, and tacit procedures. Tool protocols help, but high-quality scientific skills require local knowledge. The workbench must support institutional and disciplinary customization without losing interoperability.

The third limitation is memory governance. Persistent agents need project memory, but indiscriminate memory can preserve errors, sensitive information, and irrelevant context. Current systems can implement promotion policies and versioned summaries, yet robust semantic memory over years remains an open engineering problem. The research object should be primary; model memory should be a view.

The fourth limitation is evaluation of the system itself. Benchmarks such as PaperBench, MLE-bench, DiscoveryWorld, and laboratory suites expose parts of the problem, but a scientific workbench must also be evaluated on longitudinal correction, semantic fidelity, common-mode failure, and governance. These properties require field pilots rather than static tests.

What is possible now is therefore more significant than a better chatbot and less dramatic than a fully autonomous scientist. It is a copilot that makes research more explicit, executable, and inspectable. Its best contribution is not the number of ideas it generates but the evidential discipline it helps a laboratory maintain.

Table 7. Three claims about the future that should not be conflated

Status	Concrete interpretation
Possible now	Teams can assemble copilots that maintain specifications, retrieve literature, generate and run code, manage experiment graphs, attach provenance, conduct preflight review, and present evidence-bearing artefacts, although reliability remains domain- and task-dependent.
Probable	Serious research environments will move from transcript-centred assistants toward repositories of claims, artefacts, dependency graphs, validators, reviews, and selective recomputation; autonomy will advance fastest where feedback is executable and fast.

Status	Concrete interpretation
Desirable	Automation should increase the rate at which weak claims are rejected, important alternatives are explored, evidence is reproduced, and scarce human judgment is directed toward purpose, causal interpretation, risk, and accountable release.
Possible but undesirable	Institutions can use AI to multiply papers, reviews, benchmarks, and grant applications without increasing understanding, thereby converting generative abundance into epistemic debt and metric inflation.
Neither probable nor desirable as a universal model	One monolithic autonomous scientist that replaces domain communities, methodological pluralism, institutional responsibility, and human authority.

7.2 What is probable: 2030, 2035, and 2040

Forecasts about AI are unreliable, especially when they assume smooth scaling. The following scenarios are not predictions of model capability alone. They are conditional projections based on current technical trajectories, the economics of research, and the slow adaptation of institutions. Their purpose is to clarify what infrastructure choices matter.

By 2030, persistent scientific copilots are likely to be ordinary in computationally intensive fields. They will be connected to institutional repositories, literature services, code environments, and data platforms. A project will have a living specification and a machine-maintained map of claims, evidence, and tasks. Agents will propose and implement analyses, rerun affected branches after changes, and assemble reproducibility packages. Human researchers will review diffs and claim transitions rather than reading every generated token.

The most visible change will be that the manuscript is generated late from a maintained research state rather than used as the place where the project state is assembled. Results, methods, and citations will be synchronized with artefacts. Journals will increasingly accept or require machine-readable research packages. Automated preflight will become similar to continuous integration: a submission that lacks environment capture, claim-evidence links, or baseline accounting will fail before human review in fields where these requirements are feasible.

Coding-agent techniques will transfer directly. Laboratories will maintain project instructions, specialized skills, hooks, and eval suites. A change to a dataset or analysis function will trigger relevant tests and identify affected claims. The scientific analogue of a pull request will present not only file diffs but epistemic diffs: assumptions changed, evidence added, claim narrowed, critic disagreement unresolved. Reviewers will operate on this change set.

By 2030, the strongest systems will still be heterogeneous and supervised. Formal and computational branches will have high local autonomy. Literature and information workflows will be largely automated but audited. Laboratory planning will be common, while physical execution autonomy will remain concentrated in facilities with standardized instruments. Clinical and social research will use copilots extensively for analysis and protocol support but maintain strong human and regulatory gates.

By 2035, research workbenches are likely to coordinate networks rather than single projects. Machine-readable capability descriptions will allow systems to allocate simulations, analyses, and experiments across laboratories and cloud services. Inter-laboratory replication can be triggered automatically for claims that reach a specified stage or show high common-mode risk. Evidence objects will accumulate across institutions while preserving local access control.

Portfolio science will become more explicit. A major research programme may maintain hundreds of branches, but only a small number will receive expensive evidence. Agents will estimate expected information gain, cost, risk, and diversity. Negative results will update shared models and prevent redundant work. Funding agencies may support portfolios as versioned programmes rather than evaluate only fixed proposals and final papers.

Neuro-symbolic and domain-specific interpreters will become more important than ever-larger monolithic agents. The easiest gains from general language capability will have been captured. Reliability will depend on tool-specific validators, typed representations, causal models, proof systems, and instrument models. Scientific runtimes will route tasks according to evidence requirements. The term “agent” may become less central as adaptive components are embedded inside ordinary research infrastructure.

Peer review will be continuous for evidence objects. A paper can remain the citable public argument, but claims will receive updates from replication, reanalysis, or new evidence. Journals and repositories may display evidence maturity and unresolved dissent rather than a binary published status. Machine assistance will reduce routine checks, but specialist attention will remain scarce and may become a separately funded activity.

By 2040, limited autonomous research programmes are plausible in domains with strong evaluators, digital execution, and manageable risk. A system may maintain a long-lived objective, generate and prune branches, allocate compute, produce proofs or algorithms, and release low-risk results under a standing policy. Some autonomous laboratories may operate continuously within tightly defined chemical, materials, or biological spaces. The important qualifier is limited. The objective, risk envelope, evidence profile, and release authority will be externally governed.

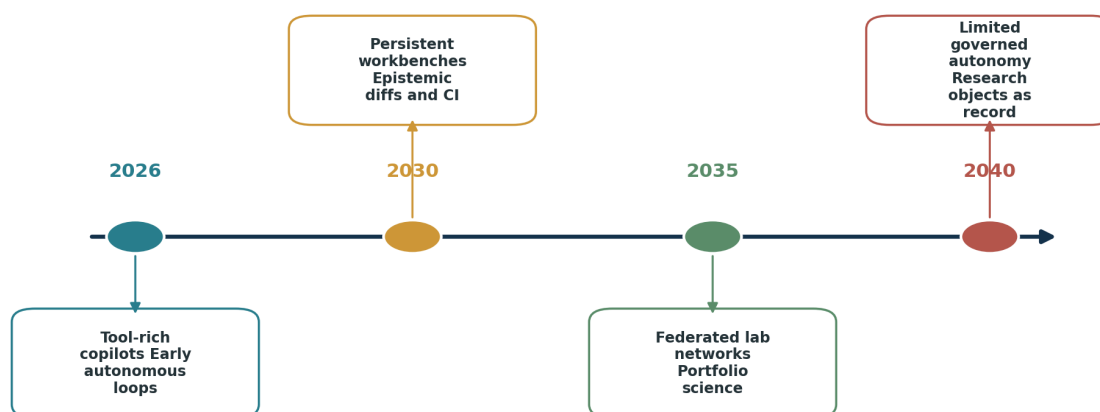
A universal autonomous scientist is less probable. Science contains too many heterogeneous regimes and contested objectives. Even very capable models will operate through institutions, instruments, data rights, and public consequences. The more plausible endpoint is institutional autonomy: a human-machine organization capable of maintaining inquiry over long periods, revising its representations, and producing evidence-bearing outputs. Intelligence resides in the architecture and governance as much as in any model.

Several developments could slow or redirect these scenarios. Model progress may plateau on long-horizon reliability. Compute and energy may constrain broad search. Data access and copyright disputes may limit literature integration. Laboratories may resist standardization. Research fraud may exploit automated infrastructure faster than governance adapts. Public backlash may restrict autonomous experimentation. Conversely, breakthroughs in theorem proving, causal modelling, robotics, or persistent memory could accelerate specific domains.

The key uncertainty is not whether AI can produce impressive scientific episodes. It is whether institutions can absorb them. Universities and journals change more slowly than software platforms. Incentives may continue to reward papers rather than evidence objects. Proprietary systems may fragment standards. Researchers may reject heavy workflow structure. The transition will likely be uneven, with leading computational and industrial laboratories adopting workbenches long before many disciplines.

The most important forecast is therefore architectural: scientific results will migrate from documents into versioned systems. This migration is already beginning through repositories, notebooks, data packages, preregistrations, and automated workflows. AI increases the pressure because the volume and speed of transformations make informal memory inadequate. Whether the systems are called scientific workbenches, research operating systems, evidence graphs, or laboratory copilots, their function will be to preserve the state of inquiry outside the model’s conversation.

A plausible maturation path, 2026-2040



The dates are conditional scenarios, not guaranteed capability milestones.

Figure 8. A plausible maturation path. Dates are conditional scenarios, not guaranteed model-capability milestones.

7.3 What is desirable: a concrete programme for executable science

Technical possibility does not determine the future worth building. A desirable automated-science system should increase the rate at which important, reliable knowledge is produced while protecting intellectual freedom, human welfare, and the plurality of scientific judgment. This section states the book's normative position as an engineering programme.

The first objective is to optimize correction rather than output. The workbench should be judged by how quickly it finds valid counterexamples, how often it demotes claims after simpler explanations, how reliably independent teams reproduce results, and how much specialist time is spent on claims that survive preliminary checks. Publication count, hypothesis volume, and model-rated novelty are secondary indicators at best.

The second objective is to preserve the distinction between proposal and judgment. Generative systems should be free to explore within bounded branches. Promotion requires new evidence types and, at consequential stages, independent critics and named authority. The generator should never be the sole certifier of its own output. Multi-agent debate is not independence unless dependencies differ materially.

The third objective is progressive formalization. Research should begin in language when concepts are open, move into controlled representations where commitments can be extracted, and use formal or deterministic methods where semantics support them. The system should not force premature closure, but it should not leave stable obligations hidden in prose. Symbolic processing should replace LLM judgment when it is faster, cheaper, and stronger.

The fourth objective is interoperability. Research objects should use open identifiers, provenance standards, portable execution descriptions, and replaceable tool interfaces. Institutions should be able to change model providers and orchestration frameworks without losing scientific memory. Domain communities should control rigor profiles and governance while sharing a minimal evidence vocabulary.

The fifth objective is layered transparency. Public claims require inspectable evidence and provenance. Private exploration requires protection. Institutional audit requires controlled access.

The workbench should preserve consequential reasons without becoming a total surveillance system. Researchers need a right to provisional error, and affected communities need a right to contest released claims.

The sixth objective is proportional governance. The evidence and authority required for a transition should depend on consequence. Low-risk conjectures can move quickly. High-stakes interventions require external validation, safety review, and accountable decision-making. Release policies should be explicit, versioned, and reversible.

The seventh objective is epistemic diversity. The portfolio should include alternative models, evaluators, data sources, and critics. The system should measure dependency diversity and common-mode risk. Funding and infrastructure should protect questions that lack abundant data or easy benchmarks. Scientific automation should not turn the training distribution of a few frontier models into the global research agenda.

These objectives define a concrete research programme. The first workstream is specification fidelity. Can a research objective and its constraints survive transformation into hypotheses, code, experiments, analyses, and release claims? Pilots should measure semantic drift and the rate at which downstream artefacts lose or strengthen commitments.

The second workstream is critic independence. Systems should be tested with seeded confounds, misleading literature, shared model dependencies, and evaluator exploits. The goal is to determine which combinations of deterministic checks, heterogeneous models, and human review detect failures. The relevant metric is not agreement but error detection under correlated pressure.

The third workstream is claim-evidence interoperability. A research object created in one workbench should be interpretable in another. Claims should retain identity across versions, and evidence should preserve scope. A later system should be able to reconstruct why a claim changed status without access to proprietary transcripts.

The fourth workstream is selective recomputation. When data, code, assumptions, or policy change, the system should identify affected nodes accurately. The experiment should measure false invalidation, missed dependencies, cost, and the rate at which regenerated outputs introduce unrelated drift.

The fifth workstream is domain validation. The architecture should be piloted in at least one formal domain, one computational empirical domain, one physical or life-science laboratory, and one domain with contested interpretation or high consequence. A system that works only in machine learning should not be presented as a general scientific runtime.

The sixth workstream is governance. Access control, decision rights, release thresholds, authorship, privacy, and stop mechanisms should be tested under realistic collaboration. The system should support disagreement, correction, and migration. Governance failures should be treated as research failures, not externalities.

The seventh workstream is human capability. Longitudinal studies should measure whether scientists become better at specification and criticism or merely more dependent on automation. Researchers should be tested on their ability to detect failures, explain assumptions, and take over when the system is wrong. Productivity measures should include validation and maintenance, not only output.

The ACHILLES agenda fits within this programme. SOP Lang Plan and Pipeline should be tested for interactive and build-like research workflows. Scientific skills should be generalized into interpreters with explicit contracts. MRP-VM should be compared against conventional orchestration on regime selection, traceability, cost, and validation. The Mirroring Pattern should be

tested on long-lived projects to determine whether specification structure improves local correction and reduces semantic drift.

A credible implementation path should begin more modestly than the rhetoric of an autonomous scientist. Between 2026 and 2030, a laboratory or research group can treat the workbench as an evidence engineer around existing human practice. The first stable layer is a repository that gives persistent identities to specifications, claims, datasets, code, protocols, environments, reviews, and decisions. The second is a dependency service that can answer which results are affected when an input or assumption changes. The third is a library of validators and interpreters whose contracts state applicability and limitations. The fourth is a promotion policy that distinguishes exploratory output from reusable knowledge and public claims. Only after these layers are reliable should broader agentic autonomy be allowed to plan expensive experiments, modify central specifications, or initiate consequential release.

This sequence is desirable because it produces value before full autonomy. Researchers benefit immediately from better context selection, reproducible builds, review preflight, explicit unresolved issues, and selective recomputation. Institutions gain a more honest account of where evidence came from and why a conclusion changed. The system can automate clerical reconstruction without pretending to automate scientific authority. Early deployments should therefore be evaluated against ordinary research practice, not against a fictional human-free laboratory. The relevant question is whether the workbench helps competent teams make fewer undetected errors, explore stronger alternatives, and maintain intelligible projects over time.

A minimal interoperable core should remain small. It needs stable identities for objects and claims; versioned links between specification, execution, evidence, and decision; typed status transitions; portable descriptions of environments and validators; explicit authority; and a way to preserve disagreement. Rich domain semantics can be layered above this core. A chemistry community may define synthesis and characterization profiles. A clinical community may define consent, cohort, endpoint, and adjudication obligations. A theorem-proving community may define trusted kernels and formalization provenance. The common infrastructure should not decide these meanings centrally. It should make them composable and inspectable.

Procurement and funding policy will be as important as model quality. Public research funders can require that consequential computational results include machine-readable claim-evidence relations and executable views without requiring universal disclosure of sensitive data. Journals can accept papers as signed releases of versioned research objects and allow post-publication replication evidence to update their status. Universities can credit the construction of reusable validators, negative-result repositories, and maintained datasets as scientific contributions. Regulators can ask not for an AI-generated explanation alone, but for the evidence state, provenance profile, unresolved objections, and named decision authority behind a consequential claim.

The workbench must also expose its own costs. Every additional validator, provenance requirement, and independent witness consumes compute, storage, specialist time, and organizational attention. Rigor cannot mean maximal logging. It should mean proportionate assurance. A low-consequence exploratory branch may retain a compact trace; a clinical recommendation or dangerous synthesis procedure may require a far deeper evidence package. The specification should make that assurance profile explicit before execution where possible, and any later relaxation should be visible as a governed change.

The most important product metric is therefore not autonomy. It is epistemic leverage: the amount of reliable correction, independent evidence, and preserved understanding obtained per unit of scarce human attention and material cost. A system that runs ten times more experiments but consumes all expert capacity in retrospective debugging has not automated research; it has

automated the creation of validation debt. A system that performs fewer experiments but finds a decisive counterexample early, preserves the failed branch, and redirects the programme may be scientifically superior. This inversion of the usual productivity narrative is central to the desirable future.

Several falsifiable predictions follow. By 2030, leading computational laboratories will maintain machine-readable claim-evidence graphs for major projects, even if standards remain fragmented. Reproducibility packages will become continuous build products rather than materials assembled after acceptance. Peer-review systems will use AI preflight widely, but fully automated editorial authority will remain exceptional and controversial. Scientific coding agents will improve measured productivity on many routine tasks, while projects without specification and validation discipline will show rising maintenance and replication debt. In formal and algorithmic domains, human researchers will increasingly contribute problem selection, representation, and evaluator design while agents conduct much of the search. In laboratory science, autonomous execution will expand mainly inside standardized facilities and narrow safety envelopes. High-stakes clinical and policy claims will retain human and institutional decision gates despite better models.

A more ambitious prediction is that the paper will cease to be the primary unit of machine-readable scientific memory. By the mid-2030s, influential results will be accompanied by persistent objects that expose claims, versions, evidence, and executable views. Citations may refer to object versions or claim identifiers as well as documents. Corrections and replications will update the evidence state without requiring a new narrative for every change.

These predictions can fail. Institutions may preserve paper-centric incentives; proprietary platforms may block interoperability; scientists may reject structured workflows; or technical systems may remain too brittle. Failure would be informative. The book's thesis is not that executable science is inevitable. It is that generative abundance makes some version of it necessary if research is to remain governable.

The desirable future is not one in which machines remove uncertainty, disagreement, or human judgment. Those are constitutive features of inquiry. The desirable future is one in which uncertainty is represented, disagreement is preserved, judgment is allocated where it matters, and evidence can travel with claims. AI should make science easier to challenge as well as easier to produce.

Table 8. A concrete research programme for executable science

Research programme	Observable success condition
Executable replication and extension	A specification-driven workbench reproduces and extends an open study with higher trace fidelity, fewer unsupported claims, faster reviewer localization, and cheaper selective recomputation than a monolithic prompt, fixed workflow, or unconstrained ReAct loop.
Scientific interpreter comparison	The same responsibility is implemented as an LLM subagent, deterministic program, solver-backed component, and hybrid unit; accuracy, cost, latency, evidence quality, and failure modes reveal when each regime should be selected.
Specification explosion	Constructive and adversarial branches discover decisive negative evidence earlier, cover more of the hypothesis space, and reduce premature commitment relative to optimizing only the initial idea.
Evidence-bearing memory	Policies that separate provisional output from promoted knowledge reduce self-confirmation, knowledge pollution, and hidden dependency drift across long projects.

Research programme	Observable success condition
Human factors and epistemic debt	Researchers using the workbench retain better understanding, detect seeded errors more often, and calibrate trust more accurately than users of an output-equivalent conversational assistant.
Laboratory or simulator integration	The architecture manages instruments, transactions, safety gates, recovery, calibration evidence, and independent checks in a real or high-fidelity environment without erasing human decision rights.

Epilogue. Science After the Manuscript

The scientific paper will survive the arrival of research agents for the same reason that source code did not eliminate design documents and legal records did not eliminate public argument. Human communities need narratives that explain what matters. A graph of claims and evidence can preserve structure, but it does not select significance. A reproducible workflow can show how an output was produced, but it cannot decide why the result deserves attention. A theorem prover can certify a statement, but it cannot determine whether the statement changes a field. The paper remains the place where researchers accept responsibility for an intelligible interpretation.

What changes is its monopoly. The manuscript can no longer carry the whole burden of scientific memory when machines generate and transform research artefacts at a speed that exceeds human reconstruction. The result must live in a larger object: versioned, evidence-bearing, and partially executable. The paper becomes a signed view over that object. A reviewer can inspect the route from claim to evidence. A replicator can rerun the relevant branch. A future agent can learn from a negative result. An institution can see who authorized release. A community can preserve disagreement without rewriting history.

This infrastructure makes science more software-like, but the analogy stops at the point where the world, interpretation, and authority resist formal closure. Software is judged against purposes that can often be stated before implementation. Research frequently changes the language of its purpose. The specification is living. A successful experiment can revise the hypothesis, the evaluator, or the question. Executable science must therefore preserve the ability to change frames rather than merely optimize within them.

That is why the central risk is not an intelligent machine but a mechanical loop. Any system can become mechanical when it continues to optimize a proxy after the proxy has lost contact with the purpose of inquiry. A model can optimize plausible prose. A benchmark community can optimize scores. A journal can optimize citation and throughput. A researcher can optimize publication count. The appropriate defence is meta-rational: representations, evaluators, and decision rights must remain revisable under evidence.

Specification-driven inquiry provides one architecture for this defence. It separates persistent intent from transient prompts. It creates a ladder from mission to claim, protocol, artefact, and release. It explodes one specification into a portfolio of constructive and adversarial branches. It places deterministic pipelines around adaptive local agents. It treats skills as interpreters that declare assumptions and evidence. It promotes claims through new kinds of support rather than through repeated self-confidence. It makes stopping, disagreement, and negative results first-class outcomes.

No single project can implement this entire vision. Industry has already provided important components: structured generation, spec-driven coding environments, agent skills, model context protocols, state graphs, tool sandboxes, and evaluation infrastructure. Science has provenance standards, workflow languages, research-object packaging, preregistration, reproducible notebooks, proof assistants, causal models, and self-driving laboratories. The ACHILLES direction proposes a unifying layer through SOP Lang and a meta-rational runtime, but that proposal must earn its place through comparative evidence. The future will be built by composing, testing, and simplifying these mechanisms, not by declaring a universal architecture in advance.

The most valuable scientific copilot will not be the one that sounds most like a scientist. It will be the one that helps a research group preserve intent, design stronger tests, expose semantic drift, reproduce results, allocate attention, and correct itself. It will know when to invoke a language

model and when to avoid one. It will not confuse an executable result with a justified claim. It will make authority visible and leave room for contest.

The human role becomes more demanding, not ceremonial. Researchers will formulate purposes, choose representations, build or select evaluators, interpret resistance from the world, and take responsibility for release. They will need to understand less of every implementation detail and more about the boundaries of each delegated regime. Institutions will need to reward maintenance, replication, negative results, and decisive criticism. Education will need to cultivate diagnostic judgment rather than routine production.

The transition is probable because the old system of record is already strained. Research projects are distributed across repositories, notebooks, data stores, instruments, and conversations. Generative systems multiply these transformations. Informal memory cannot scale. Either science develops a versioned operational substrate, or it accepts that increasingly consequential results will be produced through processes no one can reconstruct.

The desirable future is not total automation. It is accountable acceleration. Local agents explore; external evaluators resist; specialists judge bounded questions; institutions authorize consequences; and the research object remembers why claims changed. Science remains uncertain and social, but becomes more computationally articulate.

In that future, publication is not the end of a result. It is a release. Evidence continues to arrive. Claims can be narrowed, strengthened, reproduced, contested, or superseded. The durable achievement is not the frozen article but a maintained relation between question, evidence, and responsibility.

Science becomes executable when it can show not only what it concluded, but what would make it conclude otherwise.

References

Academic literature

- Abolhasani, M., and Kumacheva, E. (2023). The rise of self-driving labs in chemical and materials sciences. *Nature Synthesis*, 2, 483-492.
- Becker, J., Rush, N., Barnes, B., and Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. METR research report.
- Beurer-Kellner, L., Fischer, M., and Vechev, M. (2023). Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI). <https://doi.org/10.1145/3591300>
- Birhane, A., Kasirzadeh, A., Leslie, D., and Wachter, S. (2023). Science in the age of large language models. *Nature Reviews Physics*, 5, 277-280.
- Boiko, D. A., MacKnight, R., Kline, B., and Gomes, G. (2023). Autonomous chemical research with large language models. *Nature*, 624, 570-578. <https://doi.org/10.1038/s41586-023-06792-0>
- Burger, B., Maffettone, P. M., Gusev, V. V., et al. (2020). A mobile robotic chemist. *Nature*, 583, 237-241.
- Chan, J. S., Chowdhury, N., Jaffe, O., et al. (2024). MLE-bench: Evaluating machine learning agents on machine learning engineering. *arXiv:2410.07095*.
- Chambers, C. D. (2019). The Registered Reports revolution: Lessons in cultural reform. *Significance*, 16(4), 23-27.
- Chen, H., Xiong, M., Lu, Y., et al. (2025). MLR-Bench: Evaluating AI agents on open-ended machine learning research. *arXiv:2505.19955*.
- Coley, C. W., Thomas, D. A., Lummiss, J. A. M., et al. (2019). A robotic platform for flow synthesis of organic compounds informed by AI planning. *Science*, 365, eaax1566.
- de Moura, L., and Bjørner, N. (2008). Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, 337-340.
- Fawzi, A., Balog, M., Huang, A., et al. (2022). Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610, 47-53. <https://doi.org/10.1038/s41586-022-05172-4>
- Frazier, P. I. (2018). A tutorial on Bayesian optimization. *arXiv:1807.02811*.
- Ghareeb, A. E., Chang, B., Mitchener, L., et al. (2026). A multi-agent system for automating scientific discovery. *Nature*, 655, 497-505. <https://doi.org/10.1038/s41586-026-10652-y>
- Giannakopoulou, D., Pressburger, T., Mavridou, A., Rhein, J., Schumann, J., and Shi, N. (2020). Formal requirements elicitation with FRET. *Joint Proceedings of the REFSQ 2020 Workshops, CEUR-WS Vol. 2584*.
- Gottweis, J., Weng, W.-H., Daryin, A., et al. (2026). Accelerating scientific discovery with Co-Scientist. *Nature*, 655, 487-496. <https://doi.org/10.1038/s41586-026-10644-y>
- Hao, Q., Xu, F., Li, Y., et al. (2026). Artificial intelligence tools expand scientists' impact but contract science's focus. *Nature*, 649, 1237-1243. <https://doi.org/10.1038/s41586-025-09922-y>
- Häse, F., Roch, L. M., and Aspuru-Guzik, A. (2019). Next-generation experimentation with self-driving laboratories. *Trends in Chemistry*, 1(3), 282-291.
- Hernán, M. A., and Robins, J. M. (2020). *Causal Inference: What If*. Chapman & Hall/CRC.
- Ioannidis, J. P. A. (2005). Why most published research findings are false. *PLoS Medicine*, 2(8), e124.
- Jansen, P., Côté, M.-A., Khot, T., et al. (2024). DiscoveryWorld: A virtual environment for developing and evaluating automated scientific discovery agents. *arXiv:2406.06769*.
- Khattab, O., Singhvi, A., Maheshwari, P., et al. (2024). DSPy: Compiling declarative language model calls into self-improving pipelines. *International Conference on Learning Representations*. *arXiv:2310.03714*.
- King, R. D., Whelan, K. E., Jones, F. M., et al. (2004). Functional genomic hypothesis generation and experimentation by a robot scientist. *Nature*, 427, 247-252. <https://doi.org/10.1038/nature02236>
- King, R. D., Rowland, J., Oliver, S. G., et al. (2009). The automation of science. *Science*, 324, 85-89. <https://doi.org/10.1126/science.1165620>
- Kluyver, T., Ragan-Kelley, B., Pérez, F., et al. (2016). Jupyter Notebooks - a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing*, 87-90. <https://doi.org/10.3233/978-1-61499-649-1-87>
- Kon, P. T. J., Liu, J., Zhu, X., et al. (2025). EXP-Bench: Can AI conduct AI research experiments? *arXiv:2505.24785*.
- Krenn, M., Pollice, R., Guo, S. Y., et al. (2022). On scientific understanding with artificial intelligence. *Nature Reviews Physics*, 4, 761-769.
- Kuhn, T. (2014). A survey and classification of controlled natural languages. *Computational Linguistics*, 40(1), 121-170.
- Lakatos, I. (1978). *The Methodology of Scientific Research Programmes*. Cambridge University Press.

- Laurent, J. M., Hsieh, C., Goel, A., et al. (2024). LAB-Bench: Measuring capabilities of language models for biology research. *arXiv:2407.10362*.
- Lebo, T., Sahoo, S., McGuinness, D., et al. (2013). PROV-O: The PROV Ontology. W3C Recommendation.
- Lieder, F., and Griffiths, T. L. (2017). Strategy selection as rational metareasoning. *Psychological Review*, 124, 762-794. <https://doi.org/10.1037/rev0000075>
- Lindsay, R. K., Buchanan, B. G., Feigenbaum, E. A., and Lederberg, J. (1993). DENDRAL: A case study of the first expert system for scientific hypothesis formation. *Artificial Intelligence*, 61(2), 209-261. [https://doi.org/10.1016/0004-3702\(93\)90068-M](https://doi.org/10.1016/0004-3702(93)90068-M)
- Longino, H. E. (1990). *Science as Social Knowledge*. Princeton University Press.
- Lu, C., Lu, C., Lange, R. T., et al. (2026). Towards end-to-end automation of AI research. *Nature*, 651, 914-919. <https://doi.org/10.1038/s41586-026-10265-5>
- Madaan, A., Tandon, N., Gupta, P., et al. (2023). Self-Refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36.
- Messeri, L., and Crockett, M. J. (2024). Artificial intelligence and illusions of understanding in scientific research. *Nature*, 627, 49-58. <https://doi.org/10.1038/s41586-024-07146-0>
- Munafò, M. R., Nosek, B. A., Bishop, D. V. M., et al. (2017). A manifesto for reproducible science. *Nature Human Behaviour*, 1, 0021. <https://doi.org/10.1038/s41562-016-0021>
- Newell, A., and Simon, H. A. (1956). The logic theory machine - a complex information processing system. *IRE Transactions on Information Theory*, 2(3), 61-79. <https://doi.org/10.1109/TIT.1956.1056797>
- Nosek, B. A., Alter, G., Banks, G. C., et al. (2015). Promoting an open research culture. *Science*, 348, 1422-1425. <https://doi.org/10.1126/science.aab2374>
- Nosek, B. A., Ebersole, C. R., DeHaven, A. C., and Mellor, D. T. (2018). The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115, 2600-2606.
- Novikov, A., Vű, N., Eisenberger, M., et al. (2025). AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv:2506.13131*.
- Page, M. J., McKenzie, J. E., Bossuyt, P. M., et al. (2021). The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ*, 372, n71.
- Panapitiya, G., Saldanha, E., Job, H., et al. (2026). AutoLabs: Cognitive multi-agent systems with self-correction for autonomous chemical experimentation. *Scientific Reports*, 16, 19554. <https://doi.org/10.1038/s41598-026-45593-z>
- Park, M., Leahey, E., and Funk, R. J. (2023). Papers and patents are becoming less disruptive over time. *Nature*, 613, 138-144. <https://doi.org/10.1038/s41586-022-05543-x>
- Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*. Cambridge University Press.
- Peng, R. D. (2011). Reproducible research in computational science. *Science*, 334, 1226-1227. <https://doi.org/10.1126/science.1213847>
- Popper, K. R. (1959). *The Logic of Scientific Discovery*. Hutchinson.
- Ram, K. (2013). Git can facilitate greater reproducibility and increased transparency in science. *Source Code for Biology and Medicine*, 8, 7. <https://doi.org/10.1186/1751-0473-8-7>
- Reichenbach, H. (1938). *Experience and Prediction*. University of Chicago Press.
- Richardson, R. A. K., et al. (2025). The entities enabling scientific fraud at scale are large, resilient, and growing rapidly. *Proceedings of the National Academy of Sciences*, 122, e2420092122. <https://doi.org/10.1073/pnas.2420092122>
- Romera-Paredes, B., Barekatin, M., Novikov, A., et al. (2024). Mathematical discoveries from program search with large language models. *Nature*, 625, 468-475. <https://doi.org/10.1038/s41586-023-06924-6>
- Schmidt, M., and Lipson, H. (2009). Distilling free-form natural laws from experimental data. *Science*, 324, 81-85.
- Settles, B. (2009). *Active Learning Literature Survey*. University of Wisconsin-Madison Computer Sciences Technical Report 1648.
- Shao, Y., Jiang, Y., Kanell, T., et al. (2024). Assisting in writing Wikipedia-like articles from scratch with large language models. *Proceedings of NAACL 2024*.
- Shinn, N., Cassano, F., Gopinath, A., et al. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36.
- Skarlinski, M. D., Cox, S., Laurent, J. M., et al. (2024). Language agents achieve superhuman synthesis of scientific knowledge. *arXiv:2409.13740*.
- Smaldino, P. E., and McElreath, R. (2016). The natural selection of bad science. *Royal Society Open Science*, 3, 160384. <https://doi.org/10.1098/rsos.160384>
- Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. *Advances in Neural Information Processing Systems*, 25.

- Soiland-Reyes, S., Sefton, P., Crosas, M., et al. (2022). Packaging research artefacts with RO-Crate. *Data Science*, 5, 97-138. <https://doi.org/10.3233/DS-210053>
- Soldatova, L. N., Aubrey, W., King, R. D., and Clare, A. (2008). The EXACT description of biomedical protocols. *Bioinformatics*, 24(13), i295-i303. <https://doi.org/10.1093/bioinformatics/btn156>
- Starace, G., Jaffe, O., Sherburn, D., et al. (2025). PaperBench: Evaluating AI's ability to replicate AI research. OpenAI research report.
- Swanson, D. R. (1986). Fish oil, Raynaud's syndrome, and undiscovered public knowledge. *Perspectives in Biology and Medicine*, 30, 7-18.
- Szymanski, N. J., Rendy, B., Fei, Y., et al. (2023). An autonomous laboratory for the accelerated synthesis of novel materials. *Nature*, 624, 86-91. <https://doi.org/10.1038/s41586-023-06734-w>
- Thakkar, N., Yuksekgonul, M., Silberg, J., et al. (2026). A large-scale randomized study of large language model feedback in peer review. *Nature Machine Intelligence*, 8, 326-336. <https://doi.org/10.1038/s42256-026-01188-x>
- Trinh, T. H., Wu, Y., Le, Q. V., He, H., and Luong, T. (2024). Solving olympiad geometry without human demonstrations. *Nature*, 625, 476-482. <https://doi.org/10.1038/s41586-023-06747-5>
- Veit, W. (2020). Model pluralism. *Philosophy of the Social Sciences*, 50, 91-114. <https://doi.org/10.1177/0048393119894897>
- Wang, H., Fu, T., Du, Y., et al. (2023). Scientific discovery in the age of artificial intelligence. *Nature*, 620, 47-60.
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018.
- Wolpert, D. H., and Macready, W. G. (1997). No Free Lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1, 67-82. <https://doi.org/10.1109/4235.585893>
- Wuchty, S., Jones, B. F., and Uzzi, B. (2007). The increasing dominance of teams in production of knowledge. *Science*, 316, 1036-1039. <https://doi.org/10.1126/science.1136099>
- Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*.
- Yao, S., Yu, D., Zhao, J., et al. (2023). Tree of Thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36.

Technical systems, standards, and industrial sources

- Anthropic (2024). Building Effective Agents. Engineering note.
- Anthropic (2025-2026). Agent Skills documentation, technical webinars, Claude Code SDK, and agent evaluation guidance. Accessed July 2026.
- Amazon Web Services / Kiro (2025-2026). Kiro specification-driven development documentation, CLI documentation, and product materials. Accessed July 2026.
- Common Workflow Language Project (2024-2026). Common Workflow Language specifications and user guide.
- Elicit (2025-2026). Systematic review, evidence extraction, and PRISMA-support documentation.
- European Commission and European Research Area Forum (2026). Living guidelines on the responsible use of generative AI in research, third version, May 2026.
- FutureHouse (2025-2026). Platform, Aviary, PaperQA2, and scientific agent documentation.
- Google DeepMind (2025). AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms.
- Google Research (2025-2026). AI Co-Scientist technical materials and scientist-in-the-loop access documentation.
- Microsoft Research (2023-2026). Guidance: Structured and constrained generation documentation.
- Model Context Protocol (2024-2026). Protocol specification, architecture, SDKs, and security guidance.
- OpenAI (2025). PaperBench: Evaluating AI's ability to replicate AI research.
- OpenAI (2024). MLE-bench: Evaluating machine learning agents on machine learning engineering.
- Outlines Project (2023-2026). Structured generation and grammar-constrained decoding documentation.
- RO-Crate Research Object Community (2026). RO-Crate Metadata Specification 1.3.
- Sakana AI (2024-2026). The AI Scientist technical reports, software releases, and AI Scientist-v2 materials.
- Scite (2025-2026). Smart Citations, evidence search, and MCP integration documentation.
- W3C (2013). PROV-O: The PROV Ontology. W3C Recommendation.

Author's conceptual materials

Alboaie, S. (2026). Automated R&D and Mechanical Minds: From Generative Abundance to Specification-Driven, Evidence-Bearing Inquiry. Open letter manuscript in preparation.